

# Chapter 1

## Welcome to Sage!

Mathematics is something that we do, not something that we watch. Therefore, the best way to learn any branch of mathematics is to actually *get started* and give things a try. The same is true for learning Sage! Just connect to the Sage “Cell Server” and dive right on in, trying the examples of the next few pages.

<https://sagecell.sagemath.org/>

### 1.1. Using Sage as a Calculator

First off, you can always use Sage as a simple calculator. For example, if you type `2+3` and click “evaluate” then you will learn the answer is 5.

The expressions can become as complicated as you like. To calculate the amount on a simple interest loan at 6% per year for 90 days, and principal \$ 900, we all know the formula to be  $A = P(1 + rt)$  so we would just type in

```
900*(1+0.06*(90/365))
```

and click “evaluate”. You will learn that the answer is 913.315068493151, or \$ 913.32 after rounding to the nearest penny.

Notice that the symbol for addition is the plus sign, and the symbol for subtraction is the hyphen. The symbol for multiplication is the asterisk, and the symbol for division is the forward slash (that’s the one found with the question mark, not the backslash.) These are the same rules used by MS-Excel and many computer languages, including C, C++, Perl, Python, and Java.

For compound interest, we’d need to take exponents. The symbol for exponents is the caret, found above the number six on most keyboards. It looks like this “^”.

Let’s consider 12% compounded annually on a signature loan for 3 years, and a principal of \$ 11,000. We all know the formula to be  $A = P(1 + i)^n$ , so we would just type in

```
11000*(1+0.12)^3
```

and click “evaluate”. You will learn that the answer is 15454.2080000000, or \$ 15,454.21 after rounding to the nearest penny. By the way, instead of clicking “evaluate,” you can just press shift-enter on the keyboard, and it will do the same thing.

Warning: It is very important not to enter a comma in large numbers when using Sage. You have to type 11000 and not 11,000

For those who don’t like the caret, you can use two asterisks in a row.

```
11000*(1+0.12)**3
```

which is actually a throw-back to the historical programming language FORTRAN which was most popular<sup>1</sup> during the 1970s and 1980s.

What if you make a mistake? Let’s say that I really meant to say \$ 13,000 and not \$ 11,000. Click on the mistake, and correct it using the keyboard, and change the 11 into 13. Now click “evaluate” again, and all is well.

### Grouping Symbols:

When there are multiple sets of parentheses in a formula, sometimes mathematicians use brackets as a type of “super parentheses.” As it turns out, Sage needs the brackets for other things, like lists, so you have to always use parentheses for grouping inside of formulas.

For example, let’s say you need to evaluate

$$550 \frac{[1 - (1 + 0.05)^{-30}]}{0.05}$$

So you should not type

```
550 [ 1 - (1+0.05)^(-30) ]/0.05
```

but rather

```
550 ( 1 - (1+0.05)^(-30) )/0.05
```

where the brackets have become parentheses.

Some very old math books use braces { and } as a sort of auxiliary also to the parentheses and brackets. These too, if they are for grouping in a formula, must become parentheses. As it turns out, Sage, as well as modern mathematical books, use the braces { and } to denote sets.

By the way, the above formula was not artificial. It is the value of a loan at 5% compounded annually for 30 years, with an annual payment of \$ 550. The formula is called “the present value of an annuity.”

---

<sup>1</sup>An irrelevant historical aside: Because rewriting software is a painful and time-consuming process, many important pieces of scientific software remained written in FORTRAN until a few years ago, and likewise the same is true of business software in COBOL. Each new programming language incorporates features of the previous generation of languages, to make it easier to learn, and accordingly one sees some truly ancient seeds of dead languages once in a while.

**Three Mistakes that are 90+% of My Errors in Sage:**

There are three mistakes that I make a lot when using Sage. For example, if you want to say  $11,000x + 1200$  then you have to type

```
11000*x+1200
```

The first error that I sometimes make is that I leave out the asterisk between 11000 and  $x$ . In Sage, you must include that asterisk, as that is the symbol of multiplication. The second error that I'll often add a comma inside the 11000, but it is not acceptable in Sage to write 11,000 for 11000. The third error is that I'll have mismatched parentheses. Any Sage expression should have the same number of (s as it has of )s—no more and no less.

A more complete list is given as Appendix A, “What to Do When Frustrated,” on Page 295.

**1.2. Using Sage with Common Functions**

Now I'll discuss how Sage works with square roots, logarithms, exponentials, and so forth.

**Square Roots:**

The standard “high school” functions are also built-in. For example, type

```
sqrt(144)
```

then click “evaluate” and you'll learn that the answer is 12. From now on, I'm not going to say “click evaluate”, because repeating that might get tiresome; I'll assume that you know you have to do that. Since Sage likes exact answers, try

```
sqrt(8)
```

and get  $2\sqrt{2}$  as your answer. If you really need a decimal, try instead

```
N( sqrt(8) )
```

and obtain

```
2.82842712475.
```

which is a decimal approximation.

The `N()` function, which can also be written `n()`, will convert what is inside the parentheses to a real number, if possible. Usually that is a decimal expansion, thus unless what is inside the parentheses is an integer<sup>2</sup> then it will be, necessarily, an approximation. Sage will assume that all decimals are mere approximations, so for example

---

<sup>2</sup>To be mathematically correct, I should say “an integer or a fraction with denominator writable as a product of a power of 5 and a power of 2.” For example, 25ths, 16ths, and 80ths can be written exactly with decimals, where as 3rds, 15ths, and 14th cannot. Observe that  $25 = 5^2$  and  $16 = 2^4$  as well as  $80 = 2^4 \times 5$ . Those denominators have only 2s and 5s in their prime factorization. Meanwhile,  $3 = 3$  and  $15 = 5 \times 3$  while  $14 = 7 \times 2$ . As you can see, those denominators have primes other than 2 and 5 in their prime factorization. If you find this interesting, you should read “Using Sage to work with Integers” on Page 145.

```
sqrt(3.4)
```

will evaluate to 1.84390889145858, without need of using `N()`.

### Higher Order Roots:

Higher order roots can be calculated like exponents are. For example, to find the sixth root of 64, do

```
64^(1/6)
```

and obtain that the answer is 2.

### The Special Constants $\pi$ and $e$ :

Frequently in math we need the special constants  $\pi$  and  $e$ . It turns out that  $\pi$  is built into Sage as “pi” (both letters lower case) and  $e$  is built in as “e” (again, lower case).

You can do things like

```
numerical_approx(pi, digits=200)
```

to get many digits of pi, or likewise

```
numerical_approx(sqrt(2), digits=200)
```

to get a high-accuracy expansion of  $\sqrt{2}$ . In this case, we get 200 digits. You can also abbreviate with

```
N(sqrt(2), prec=200)
```

as we did earlier. That’s what the `N()` or `n()` function (they are the same) is an abbreviation of, namely “numerical approx.”

### Tab Completion:

Now would be a good time to explain a neat feature of Sage. If you are typing a long command, like “`numerical_approx`” and part way through you forget the exact ending (is it approximation? approximations? appr?) then you can hit the tab button. If only one command in the entire library has that prefix, then Sage will fill in the rest for you. If it does not, then you get a list of suggestions. It is a very useful feature when you cannot remember exact commands. We will see other examples of built-in help features in Section 1.10 on Page 47.

There’s another keyboard shortcut that I like. While I mentioned it before, if you are tired of clicking “evaluate” all the time, you can just press Shift and Enter at the same time. This does the same thing as clicking “Evaluate.”

### Is Sage Case-Sensitive?

You’ve probably had a situation in life where you entered your password correctly, but discovered that it was rejected because you had the wrong capitalization. For example, perhaps you’ve left the “CAPS-LOCK” key on. In any event, Sage does think of `Pi` as different from `pi` and `Sin` as different from `sin`.

The only four exceptions known to me are  $i$  and `n()`, as well as `True` and `False`. We will not make extensive use of `True` and `False` until Chapter 5, but it is harmless to mention that you can also enter them as `true` and `false`, and Sage recognizes these as the same. You may have heard that  $\sqrt{-1}$  is often referred to as “the imaginary number” and is denoted  $i$ .

Next, you can represent  $\sqrt{-1}$  as either `i` or `I` and either way Sage will know what you meant, namely  $\sqrt{-1}$ . If you’ve never heard of imaginary numbers or  $i$ , then you can safely ignore this fact. Lastly, you can write `n(sqrt(2))` or `N(sqrt(2))` and Sage treats those as identical.

So far as I am aware, these easements only apply to  $\sqrt{-1}$  and `n()`, as well as `True` and `False` as it turns out; in all other cases, capitalization matters.

### Exponentials:

Just as

`2^3`

gives you  $2^3$  and likewise

`3^3`

gives you  $3^3$ , if you want to say  $e^3$  then just say

`e^3`

and that’s fine. Or for a decimal approximation you can do

`N(e^3)`

Also, it is worth mentioning sometimes books will write

$$\exp(5 \cdot 11 + 8) \text{ instead of } e^{5 \cdot 11 + 8}$$

and Sage thinks that’s just fine. For example,

`exp(5*11+8) - e^(5*11+8)`

evaluates to 0. Don’t forget the asterisk between the 5 and the 11.

### Logarithms:

Of course, Sage knows about logarithms—but there’s a problem. There are several types of logarithm, including the common logarithm, the natural logarithm, the binary logarithm, and the logarithm to any other base you feel like using. In high school “log” refers to the common logarithm, and “ln” to the natural logarithm. However, after calculus “log” refers to the natural logarithm. Since Sage was mainly meant for university-and-higher level work, then it is only natural they chose to use the natural logarithm for “log.”

So to find the natural logarithm of 100 type

`N( log(100) )`

for the common logarithm of 100 type

`N( log(100,10) )`

and for the binary logarithm of 100 type

`N( log(100,2) )`  
 which of course generalizes. For example, to find the logarithm of 100 taken base 42, type

`N( log(100,42) )`

Note that Sage is quite good at getting exact answers. Try  
`log ( sqrt(100^3) , 10)`  
 and you will obtain 3, the exact answer.

### A Financial Example:

Suppose you deposit \$ 5000 in an account that earns 4.5% compounded monthly. Perhaps you are curious when your total will reach \$ 7000. Using the formula  $A = P(1+i)^n$ , where  $P$  is the principal,  $A$  is the amount at the end,  $i$  is the interest rate per month, and  $n$  is the number of compounding periods (number of months), we have

$$\begin{aligned} A &= P(1+i)^n \\ 7000 &= 5000(1+0.045/12)^n \\ 7000/5000 &= 5000(1+0.045/12)^n \\ 1.4 &= (1+0.045/12)^n \\ \log 1.4 &= \log(1+0.045/12)^n \\ \log 1.4 &= n \log(1+0.045/12) \\ \frac{\log 1.4}{\log(1+0.045/12)} &= n \end{aligned}$$

And so, we type into Sage  
`log(1.4)/log(1+0.045/12)`  
 and get the response 89.8940609330801, so that we know 90 months will be required. Therefore we write the answer 90 months, or even better, 7 years and 6 months.

This is an example of a computer-assisted solution to a problem. In addition to that approach, Sage is also willing to solve the problem from start to finish, with no human intervention. We'll see that on Page 46.

### Pure Mathematics and Square Roots:

Here are a couple of notes about square roots that would appeal to math majors, or anyone who wants to work with the complex numbers. Most students will want to skip this subsection, as well as the next one on complex numbers, and continue with either "Using Sage for Trigonometry" on Page 7, or with "Using Sage to Graph 2-Dimensionally" on Page 8.

In the spirit of absolute pomposity, you may know that  $(-2)^2 = 4$  as well as  $2^2 = 4$ . So if you want *all* the square roots of a number you can do

`sqrt(4,all=True)`  
 to obtain

`[2, -2]`

where the square brackets indicate a list. Any sequence of numbers separated by commas and enclosed in square brackets is a list. We'll see other examples of lists throughout the book.

### A Taste of the Complex Numbers:

If you know about complex numbers, you can do

`sqrt(-4, all=True)`

to obtain

`[2*I, -2*I]`

where the capital letter I represents  $\sqrt{-1}$ , the imaginary constant. As I mentioned before, you can use the lowercase `i` instead of the capital one, if you prefer.

While Sage is quite good at complex analysis, and can produce some rather lovely plots of functions of a complex variable, we will not go into those details here. The color plots of complex-value functions will be covered in the electronic-only online appendix to this book "Plotting in Color, in 3D, and Animations," available on my webpage [www.gregorybard.com](http://www.gregorybard.com) for downloading.

## 1.3. Using Sage for Trigonometry

Some students are unfamiliar with trigonometry, or are in a course that has nothing to do with trigonometry. If so, then they may confidently skip this section and jump to the next one, which begins on Page 8.

The commands for trigonometry in Sage are very intuitive but it is important to remember that Sage works in radians. So if you want to know the sine of  $\pi/3$ , you should type

`sin(pi/3)`

and you will get the answer `1/2*sqrt(3)`. This is an exact answer, rather than a mere decimal approximation. You will find that Sage is very oriented toward exact rather than approximate answers. Sometimes this is irritating, because if you ask for the cosine of  $\pi/12$ , then you would type

`cos(pi/12)`

and obtain `1/12*(sqrt(3) + 3)*sqrt(6)` which is especially unsatisfying if you want a decimal. Instead, if you type

`N(cos(pi/12))`

then you will obtain `0.965925826289`, a rather good decimal approximation.

You will discover that Sage is fairly savvy when it comes to knowing when functions will go wrong. In particular, just try evaluating tangent at one of its asymptotes. For example,

`tan(pi/2)`

will produce the helpful answer "Infinity." The "rare" or "reciprocal" trigonometric functions of cotangent, secant, and cosecant, which are important

in calculus but annoying on hand-held calculators, are built into Sage. They are identified as `cot`, `sec`, and `csc`.

The inverse trigonometric functions are also available. They are used just like the trigonometric functions. For example, if you type

```
arcsin(1/2)
```

you will obtain

```
1/6*pi
```

as expected. Likewise

```
arccos(1/2)
```

produces

```
1/3*pi
```

The usual abbreviations are all known by and used by Sage. Here is a complete list:

| Math Notation: | Long-form Command:     | Short-form Command:  |
|----------------|------------------------|----------------------|
| $\sin^{-1} x$  | <code>arcsin(x)</code> | <code>asin(x)</code> |
| $\cos^{-1} x$  | <code>arccos(x)</code> | <code>acos(x)</code> |
| $\tan^{-1} x$  | <code>arctan(x)</code> | <code>atan(x)</code> |
| $\cot^{-1} x$  | <code>arccot(x)</code> | <code>acot(x)</code> |
| $\sec^{-1} x$  | <code>arcsec(x)</code> | <code>asec(x)</code> |
| $\csc^{-1} x$  | <code>arccsc(x)</code> | <code>acsc(x)</code> |

You can also use Sage to graph the trigonometric functions. We'll do that in the section entitled "Using Sage to Graph 2-Dimensionally," on Page 11.

### Converting between Degrees and Radians:

We all remember from trigonometry class that to convert radians to degrees, just multiply by  $180/\pi$ , and to convert from degrees to radians, just multiply by  $\pi/180$ . Accordingly, here's what you type, to convert  $\pi/3$  to degrees:

```
(pi/3) * (180/pi)
```

produces 60 while

```
60 * (pi/180)
```

produces  $1/3\pi$ .

The way I like to remember this technique is that a protractor is 180 degrees, and the word "protractor" begins with "p," while  $\pi$  is the Greek "p." For example, 90 degrees is half a protractor, and 60 degrees is a third of a protractor. That's why I can remember that they are  $\pi/2$  and  $\pi/3$ . Likewise, if I see  $\pi/6$ , then I know that this is one-sixth of a protractor or 30 degrees.

## 1.4. Using Sage to Graph 2-Dimensionally

My favorite shape is the parabola, so let's start there. Type

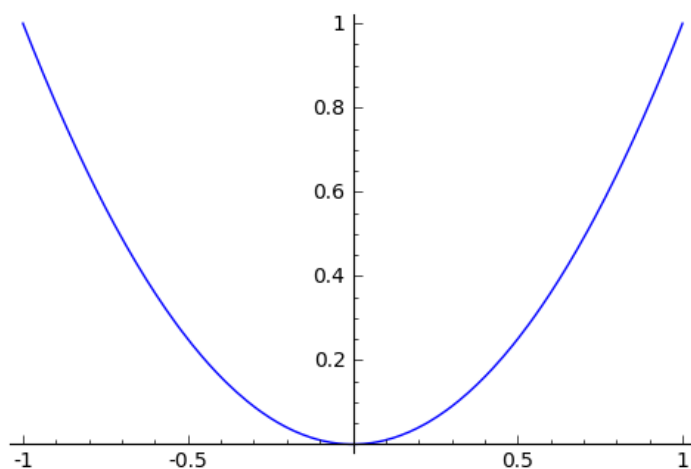


```
plot(x^2)
```

and you get a lovely plot of a parabola going in the range

$$-1 < x < 1$$

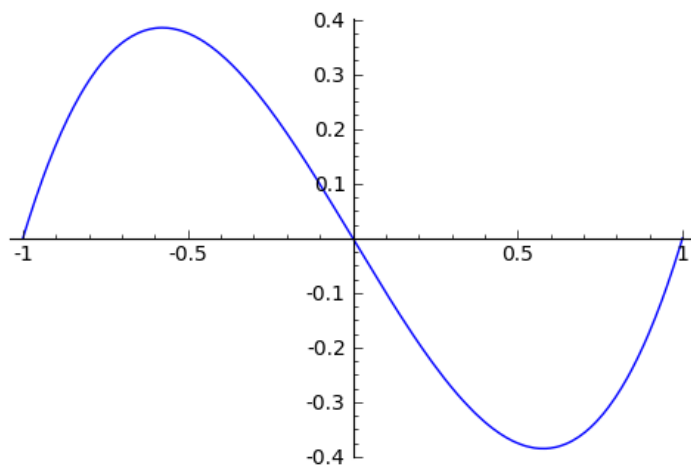
which is the default range for the `plot` command. It will bound the  $y$ -values to whatever is needed to show those  $x$ -values. Here's a screenshot of what you should see:



Likewise you can do

```
plot(x^3-x)
```

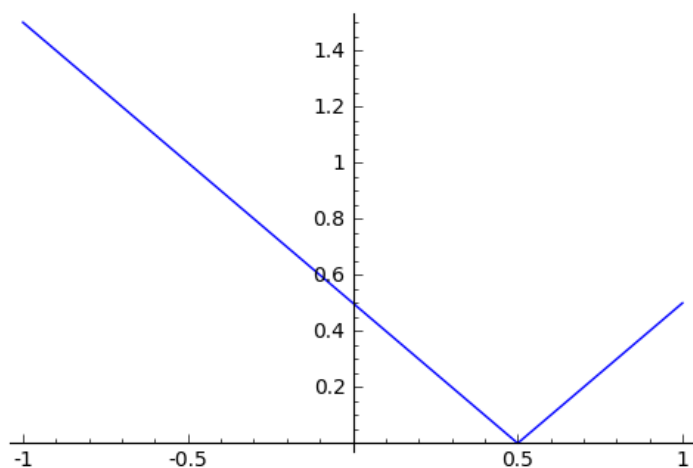
which is nice and visually appealing, as you can see:



For  $|x - 1/2|$  you can do

```
plot(abs(x-1/2))
```

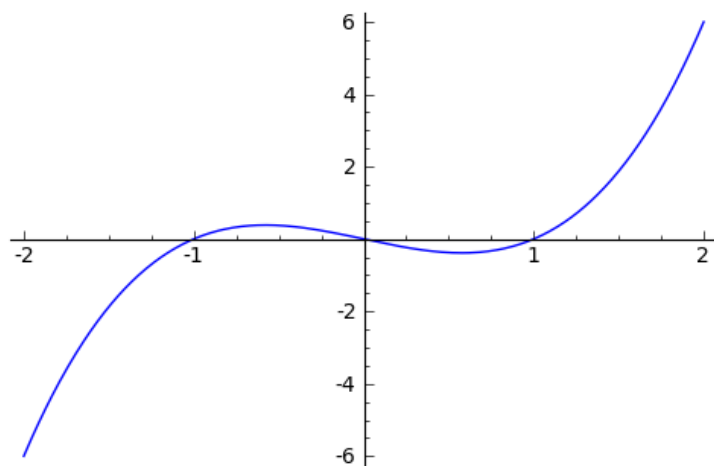
which produces the following screenshot



What if you wanted a different  $x$  range? For example, to graph in  $-2 < x < 2$  you would type

```
plot(x^3-x, -2, 2)
```

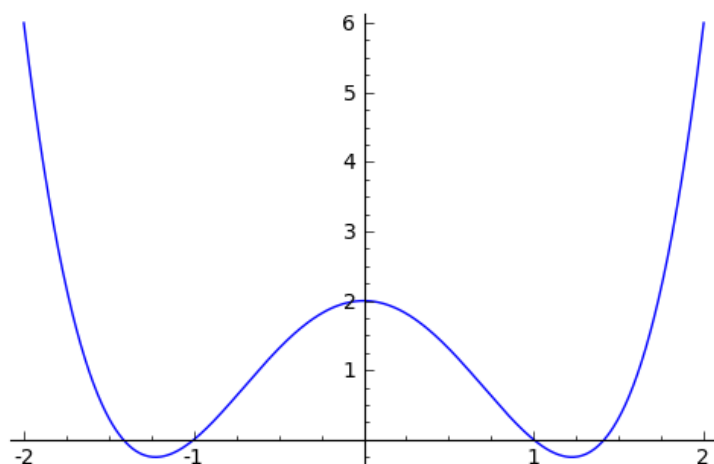
and you get the desired graph, namely:



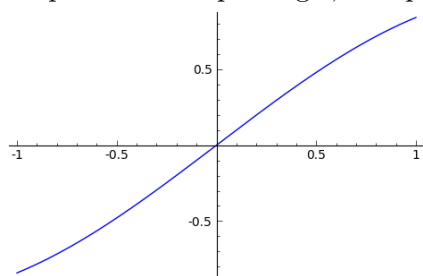
Now would be a good time to mention that you can save any of these plots to a file after having Sage generate them. Just right-click on the displayed graph in your web browser, and save the file to your computer. You can then import the image into any report or paper that you might be writing. A very cool graph is

```
plot(x^4 - 3*x^2+2,-2,2)
```

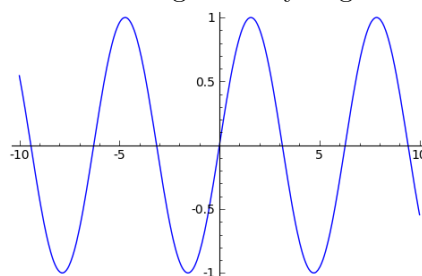
but notice the asterisk between 3 and  $x$  in “ $3*x$ ”. You will get an error if you leave that out! The plot is



Another minor sticking point is that for  $y = \sin(t)$  you have to say  
`plot(sin(x))`  
 or better yet  
`plot(sin(x), -10, 10)`  
 because plot is not expecting  $t$ , it expects  $x$ . The images that you get are



`plot(sin(x))`



`plot(sin(x), -10, 10)`

In fact if you were to type  
`plot(sin(t))`  
 you would see

`NameError: name 't' is not defined`

because in this case, Sage does not know what  $t$  means. An alternative way of handling this is given on Page 104.

#### 1.4.1. Controlling the Viewing Window of a Plot

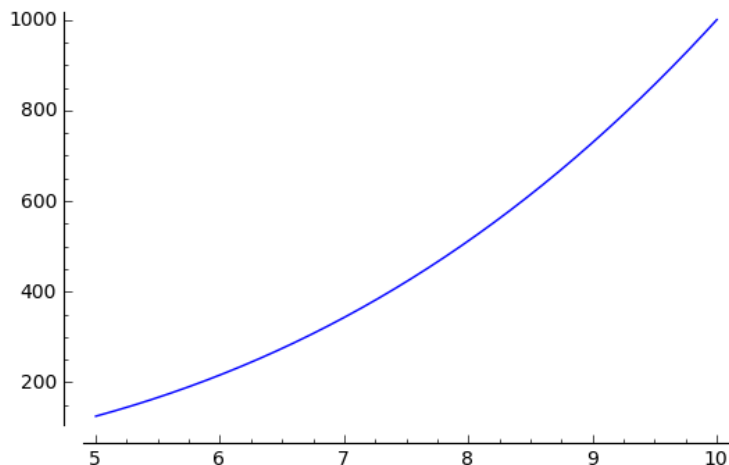
Much of the time, the default viewing window of a graph is going to be exactly what you wanted—but not always. The most common exception is a function with vertical asymptotes. Here we are going to discuss how to adjust the viewing window.

**Going “Off the Scale”:**

Consider the plot of  $x^3$  from  $x = 5$  to  $x = 10$ , which is given by

```
plot(x^3, 5, 10)
```

and as you can imagine, the function goes from  $5^3 = 125$  up to  $10^3 = 1000$ , thus the origin should appear very far below the graph. This is the plot:

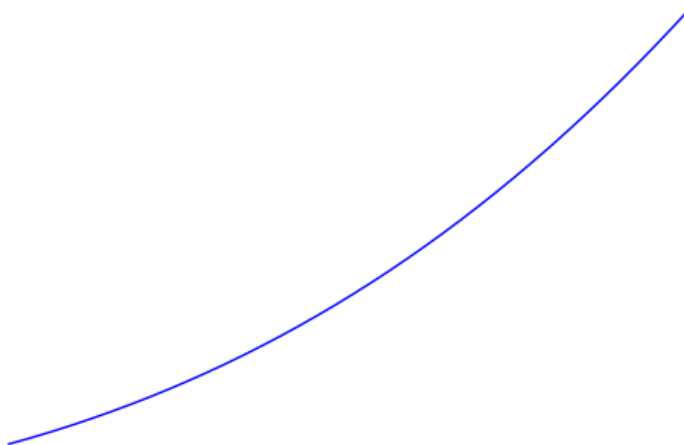


Your hint that the location of the x-axis in the display is not where it would be normally is that the axes do not intersect. This is to tell you that the origin is far away. When the axes do intersect on the screen, then the origin is (both in truth and on the screen) where they intersect.

Also, if you want to, you can hide the axes with

```
plot(x^3, 5, 10, axes=False)
```

and that produces



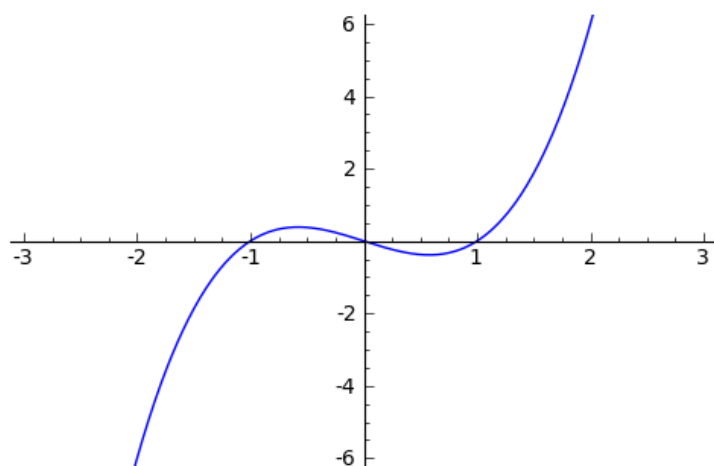
which is considerably less informative.

**To Force the Y-Range of a Graph:**

If, for some reason, you want to force the  $y$ -range of a graph to be constrained between two values, you can do that. For example, to keep  $-6 < y < 6$ , we can do

```
plot( x^3-x, -3, 3, ymin = -6, ymax = 6)
```

in order to obtain

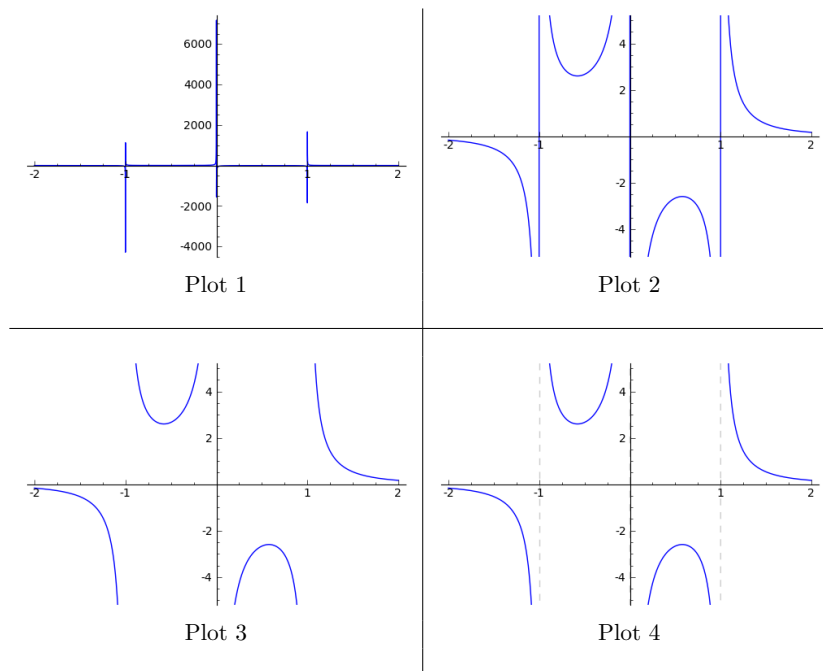


This is important, because normally Sage wants to show you the entire graph. Therefore, it will make sure that the  $y$ -axis is tall enough to include every point, from the maximum to the minimum. For some functions, either the maximum or the minimum or both could be huge.

**Plots of Functions with Asymptotes:**

The way that Sage (and all computer algebra tools) computes the plot of a function is by generating a very large number of points in the interval of the  $x$ s, and evaluating the function at each of those points. To be precise, if you want to graph  $f(x) = 1/x^2$  between  $x = -4$  and  $x = 4$ , the computer might pick 10,000 random values of  $x$  between  $-4$  and  $4$  find the  $y$  values by plugging them into  $f(x)$  and then finally drawing the dots in the appropriate spots on the graph.

So if the graph has a vertical asymptote, then near that asymptote, the value will be huge. Because of this, when you graph a rational function, be sure to restrict the  $y$ -values. For example, compare the following:

**Plot 1:**

```
plot(1/(x^3-x), -2, 2)
```

**Plot 2:**

```
plot(1/(x^3-x), -2, 2, ymin = -5, ymax = 5)
```

**Plot 3:**

```
plot(1/(x^3-x), (x,-2, 2), detect_poles=True,
      ymin = -5, ymax = 5)
```

**Plot 4:**

```
plot(1/(x^3-x), (x,-2, 2), detect_poles='show',
      ymin = -5, ymax = 5)
```

As you can see, the first is a disaster. The second one cuts off the very-high and very-low  $y$ -values, but it keeps trying to connect the various “limbs” of the graph. When you set “detect poles” to `True`, then it will figure out that the pieces are not connected.

One minor point remains. Did you notice in the four plots above, that `'show'` is in quotes, but `True` is not in quotes? That's because `True` and `False` occur so often in math, that they are built-in keywords in Sage. However, the `'show'` occurs less often, and therefore is not built in, and we must put it in quotes.

#### 1.4.2. Superimposing Multiple Graphs in One Plot

Now we're going to see how to superimpose plots on each other. It turns out that Sage thinks of this as “adding” the plots together, using a plus sign.

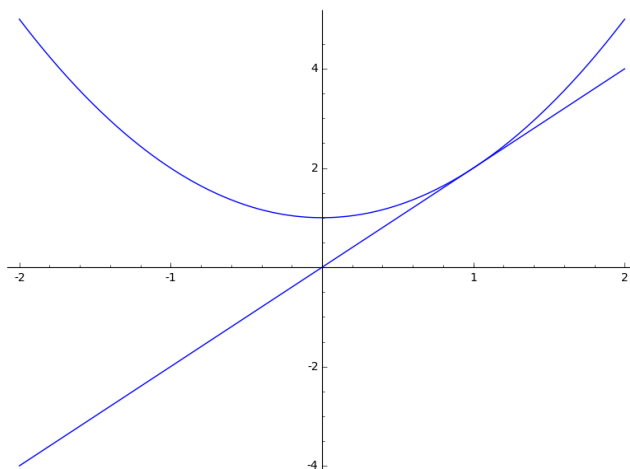
**An Example from Calculus: The Tangent Line**

Suppose one wants to draw a picture of  $f(x) = x^2 + 1$  over the interval  $-2 < x < 2$  and the tangent line to that parabola at  $x = 1$ . Because  $f(1) = 2$  and  $f'(1) = 2$ , we know the line has to go through the point  $(1, 2)$  and will have slope 2. It is not hard to compute that the equation of that line is  $y = 2x$ . The trick is that we want to graph them both at the same time, in the same picture.

The command for this will be

```
plot( 2*x, -2, 2 ) + plot( x^2+1, -2, 2 )
```

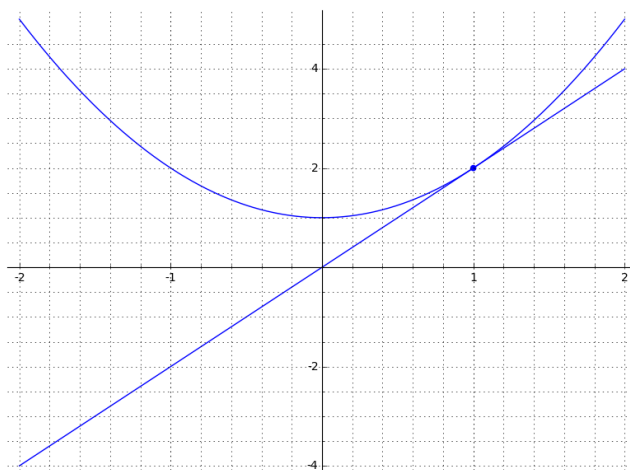
and we will get the image:



As you can see, adding the two plots makes a super-imposition. The plus sign tells Sage to draw the two curves on top of each other. Now, we can do better by adding a dot at the point of tangency—the point  $(1, 2)$  and perhaps some gridlines. The command for that will be

```
plot( 2*x, -2, 2, gridlines='minor' ) + plot( x^2+1, -2, 2 ) +  
point( (1,2), size=30 )
```

and we will get the image:



As you can see, we added the point using the `point` command and this too was superimposed using the plus sign. The `gridlines='minor'` gave us the very satisfactory grid which can be so useful in preparing nice graphs that are readable, for lab reports, classroom use, or papers for publication. We will discuss other ways of making gridlines, as well as other ways of annotating a graph, in Section 3.1 on Page 91. This specific graph will appear in a highly annotated way on Page 95.

By the way, it is important that the above command, adding the two plots plus a point, must be all on the same line. One cannot have a line break. I cannot typeset it on one line because the page I am typing on is not infinitely wide. Be sure there are no line breaks when you type those commands in.

### A More Intricate Example:

Suppose you were investigating polynomials, and you wanted to know the effect of varying the last numeral in

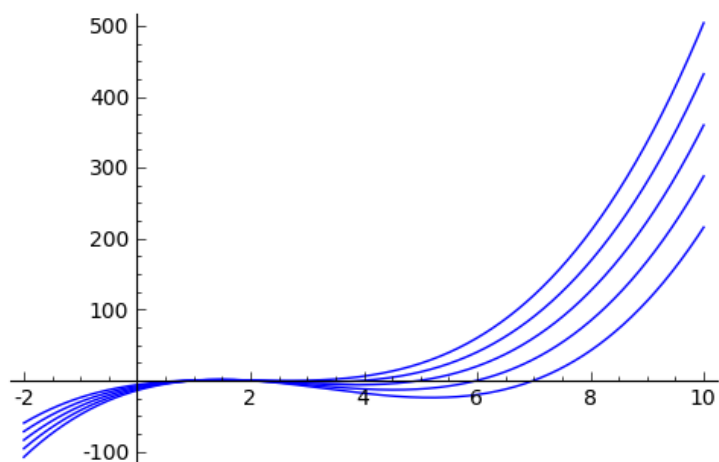
$$y = (x - 1)(x - 2)(x - 5)$$

on its graph. (In other words, you want to know what happens if you change the “5” in that equation to other numbers.) Then you could consider looking at  $(x - 3)$  and  $(x - 4)$  as replacements for the  $(x - 5)$ , as well as  $(x - 6)$  and  $(x - 7)$ . This could be done by

```
plot((x-1)*(x-2)*(x-3), -2, 10) + plot((x-1)*(x-2)*(x-4), -2,
10) + plot((x-1)*(x-2)*(x-5), -2,10) + plot((x-1)*(x-2)*(x-6),
-2, 10) + plot((x-1)*(x-2)*(x-7), -2, 10)
```

where I chose the domain,  $x = -2$  until  $x = 10$ , arbitrarily. By the way, when you enter that previous command in, it is important to note that it must be on one (very long) line. If done correctly, we obtain

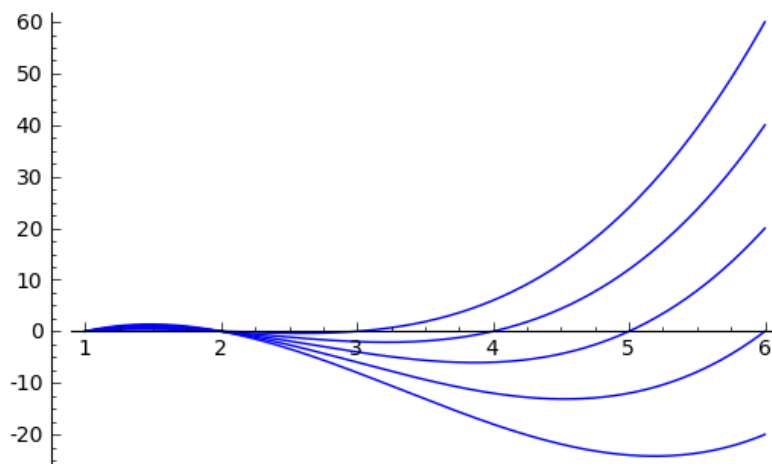




It seems that I should zoom in a bit, to see more detail. If I wanted to change the domain from  $-2 < x < 10$  to perhaps  $1 < x < 6$ , I would have to change each of the -2s into 1s and each of the 10s into 6s. So I would type

```
plot((x-1)*(x-2)*(x-3), 1,6) + plot((x-1)*(x-2)*(x-4), 1,
6) + plot((x-1)*(x-2)*(x-5), 1,6) + plot((x-1)*(x-2)*(x-6), 1,
6) + plot((x-1)*(x-2)*(x-7), 1,6)
```

and obtain as a result



There is an important issue here. We have to be careful to make the changes correctly, and that somewhat challenging requirement is made much worse because the code is hard to read. However, we will soon learn how to make the code more readable, and this will also make the code easier to change.

In any case, now I can see better what that last numeral does. It controls the location, or  $x$ -coordinate, of the last time that the curve crosses the  $x$ -axis. As you can see, the plus sign among the `plot` commands means to superimpose them.

There is a technicality here. You must not break a newline among the plots and plus signs. Therefore, if you have problems getting those last two commands to work, then check to see that you have no accidental “line breaks” in the command. For Sage, these commands must be entered without a line break, kind of like typing a long paragraph in an ordinary word processor—you can wrap around the end of the line, but you cannot insert a line break. This is an example of code that is functional, but not easy to read, nor change, and certainly not easy to understand. We can fix this, with a bit of effort. The various plot commands and their plus signs must be strung together, like a paragraph, wrapping naturally from one line to the next. You do not use the enter key to put each plot on its own line. Thus, you cannot do:

```
plot( (x-1)*(x-2)*(x-3), 1, 6)
+ plot( (x-1)*(x-2)*(x-4), 1, 6)
+ plot( (x-1)*(x-2)*(x-5), 1, 6)
+ plot( (x-1)*(x-2)*(x-6), 1, 6)
+ plot( (x-1)*(x-2)*(x-7), 1, 6)
```

which Sage thinks is five separate commands, instead of one big command.

However, there is a nice way out of this dilemma. You can type the following, which produces the desired graph.

```
P1 = plot( (x-1)*(x-2)*(x-3), 1, 6)
P2 = plot( (x-1)*(x-2)*(x-4), 1, 6)
P3 = plot( (x-1)*(x-2)*(x-5), 1, 6)
P4 = plot( (x-1)*(x-2)*(x-6), 1, 6)
P5 = plot( (x-1)*(x-2)*(x-7), 1, 6)
```

```
P = P1 + P2 + P3 + P4 + P5
P.show()
```

I’m sure we can all agree that this is much more readable code.

### Further Graphing & Plotting Topics:

We have now only scratched the surface of plotting in Sage. There is a rich library of types of plots and graphs which Sage can produce for you. An entire chapter of this book is dedicated to “Advanced Plotting Techniques” beginning on Page 91. If you’re curious about how Sage actually generates the images, we talk about that on Page 267, but you might have to read most of Chapter 5 to be able to follow that discussion.

By the way, you do not need to read that entire chapter in order. You can simply use the index or table of contents to select exactly which type

of graph or plot you would like, and just read that tiny subsection of that chapter.

## 1.5. Matrices and Sage, Part One

In this section we'll learn how to solve linear systems of equations using matrices in Sage. This section assumes that you've never worked with matrices before in any way, or alternatively, that you have forgotten them. Experts in linear algebra can skip to the Section 1.5.3 on Page 22. On the other hand, some readers will have no specific interest in matrices, and can therefore skip to Section 1.6 on Page 30, where they will learn how to define their own functions in Sage.

### 1.5.1. A First Taste of Matrices

Let us suppose that you wish to solve this linear system of equations:

$$\begin{aligned} 3x - 4y + 5z &= 14 \\ x + y - 8z &= -5 \\ 2x + y + z &= 7 \end{aligned}$$

First you would convert those equations into the following matrix:

$$A = \left[ \begin{array}{ccc|c} 3 & -4 & 5 & 14 \\ 1 & 1 & -8 & -5 \\ 2 & 1 & 1 & 7 \end{array} \right]$$

Notice that the coefficients of the  $x$ s all appear in the first column; the coefficients of the  $y$ s all appear in the second column; the coefficients of the  $z$ s all appear in the third column. The fourth column gets the constants. Thus we have encapsulated and abbreviated all of the information of the problem. Furthermore, observe that additions are represented by a positive coefficient, and subtractions by a negative coefficient. By the way, the vertical line between the third and fourth column is just decorative—its purpose is to show that the fourth column is “special” in that it contains numbers, whereas the other columns represent the coefficients of the variables  $x$ ,  $y$ , and  $z$ .

Matrices have a special form called “Reduced Row Echelon Form” often abbreviated as RREF. The RREF is, from a certain perspective, the simplest description of the system of equations that is still true. The Reduced Row Echelon Form of this matrix  $A$  is

$$\left[ \begin{array}{ccc|c} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{array} \right]$$

We can translate this literally as

$$\begin{aligned} 1x + 0y + 0z &= 3 \\ 0x + 1y + 0z &= 0 \\ 0x + 0y + 1z &= 1 \end{aligned}$$

or in simpler notation

$$x = 3 \qquad y = 0 \qquad z = 1$$

which is the solution. You should take a moment now, plug in these three values, and see that it comes out correct.

### 1.5.2. Complications in Converting Linear Systems

You might be looking at the previous discussion and imagine that what we're doing is a kind of silver-bullet, quickly capable of solving any of a large number of extremely tedious problems that take (with a pencil) a very long time.

Indeed, matrix algebra is a silver bullet, now that we have computers. Prior to the computer, large linear algebra problems were considered extremely unpleasant. Yet, now that we have computers, and because computers can carry out these operations essentially instantly, this topic is a great way to rapidly solve enormous systems of linear equations. In turn, skilled mathematicians can address important problems in science and industry, because real-world problems often have many variables.

This topic, however, is better described as “semi-automatic” rather than “automatic.” The reason I say that is because the human must be very careful to first convert the word problem into a system of linear equations. We will not cover that here. The second step is to convert that system into a matrix, and that isn't quite trivial. In fact, it is often the case that student errors occur in this step, and so we will invest a bit of time with a detailed example.

Consider this system of equations:

$$\begin{aligned} 2x - 5z + y &= 6 + w \\ 5 + z - y &= 0 \\ w + 3(x + y) &= z \\ 1 + 2x - y &= w - 3x \end{aligned}$$

As it turns out, this system of equations has the following traps:

- In the first equation, the variables are out of order, which is extremely common. Very often, both students and professional mathematicians will fail to notice this, and put the wrong coefficients in the wrong places. You can use any ordering that you want, so long as you use the same ordering for every equation. However, to avoid making an error, mathematicians usually put the variables in alphabetical order.

- Also in the first equation, the  $w$  is on the wrong side, and we must move it to the left of the equal sign. It is necessary that all the variables end up on one side of the equal sign, and all the numbers on the other side of the equal sign. With these two repairs, the first equation is now

$$-w + 2x + y - 5z = 6$$

- The second equation has the constant on the wrong side, and so we must move it across the equal sign, remembering to negate it.
- As if that were not enough, both  $x$  and  $w$  are missing in the second equation. We treat this as if the coefficients were zero. With these two repairs, the second equation is now

$$0w + 0x - y + z = -5$$

- The third equation has parentheses—that's not allowed. We have to remember that  $3(x + y) = 3x + 3y$ .
- Also in the third equation, the  $z$  is on the wrong side. We must take care to change  $+z$  into  $-z$  when crossing the equal sign.
- A third defect in the third equation is that there is no constant. We treat this as a constant of zero. Correcting these three issues, the third equation becomes

$$w + 3x + 3y - z = 0$$

- Now the real delinquent is the fourth equation. We have  $w$  on the wrong side, and what is worse is that there are two occasions of  $x$ . When we move the  $-3x$  from the right of the equal sign to the left, it becomes  $+3x$  and combines with the  $+2x$  that was already there, to form  $5x$ . As if that were not enough, the constant is on the wrong side. Last but not least,  $z$  is entirely absent. Correcting all these defects, we have

$$-w + 5x - y + 0z = -1$$

With these (modified) equations in mind, we have the matrix:

$$B = \left[ \begin{array}{cccc|c} -1 & 2 & 1 & -5 & 6 \\ 0 & 0 & -1 & 1 & -5 \\ 1 & 3 & 3 & -1 & 0 \\ -1 & 5 & -1 & 0 & -1 \end{array} \right]$$

The RREF of that matrix would be tedious to find by hand. However, it would be far worse to solve that system of equations by hand without matrices, using ordinary algebra. However, using Sage, we will easily compute the RREF in the next subsection. For now, it turns out that the answer is

$$w = -107/7 \quad x = -12/7 \quad y = 54/7 \quad z = 19/7$$

which you can easily verify now if you like.

### 1.5.3. Doing the RREF in Sage

This subsection is going to tell you how to compute the RREF of a matrix, presumably with the goal of solving some linear system of equations. Doing this in Sage requires only four steps, the last of which is optional. First, we're going to define the matrix

$$A = \left[ \begin{array}{ccc|c} 3 & -4 & 5 & 14 \\ 1 & 1 & -8 & -5 \\ 2 & 1 & 1 & 7 \end{array} \right]$$

which came from the previous subsection. As you can see, it has 3 rows and 4 columns. We'll do that with the Sage command

```
A = matrix( 3, 4, [ 3, -4, 5, 14, 1, 1, -8, -5, 2, 1, 1, 7 ] )
```

The key to understanding that Sage command is to see that the first number is the number of rows, followed by the number<sup>3</sup> of columns. Then follows the list of entries, separated by commas, and enclosed in brackets.

By the way, keep this definition of the matrix  $A$  in the Sage Cell for as long as you are working with it. Do not erase it until you are done with it. Every time you hit evaluate, you are sending whatever is in the Sage Cell to the Sage Cell Server for evaluation. The server won't remember the definition of the matrix  $A$  if you remove it. In contrast, SageMathCloud (See Appendix B), has much more permanent memory.

Notice that Sage does not respond when this command is given. That's because we told it what the matrix  $A$  is but we didn't give it anything to do to  $A$ .

The second step is to verify that the matrix you typed was the matrix that you had hoped to enter. This step is not avoidable, as typos are extremely common. To do this, just type `print A` on a line by itself, below the definition of the matrix  $A$ . Then click Evaluate. The matrix is displayed and you can verify that you have typed what you had wanted to type.

The third step is that you want to compute the RREF or "Reduced Row Echelon Form." This is done with the command

```
print A.rref( )
```

and Sage tells you the Reduced Row Echelon Form, or

$$\left[ \begin{array}{ccc|c} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{array} \right]$$

The fourth, and optional step, is to check your work, but I highly recommend it! First we type (on its own line)

```
print 3*3 + -4*0 + 5*1
```

and learn that the answer is 14, as expected. Next we type

---

<sup>3</sup>This is how I remember: Columns, such as on a bank, Congress, or a Greek temple, are vertical. Rows, by process of elimination, are horizontal. But do we enter the rows first, or the columns first? I always think "RC Cola," a minor brand of soda, which reminds me rows-first-columns-second. Thankfully, there does not exist a "CR Cola."

```
print 1*3 + 1*0 - 8*1
```

and learn that the answer -5, as expected. The last equation is checked similarly.

Note that it is crucial to check all three equations! Otherwise, you'd fail to detect the "imposter" solution

$$x = 30 \qquad y = 29 \qquad z = 8$$

which satisfies the first two equations, but not the third one.

Now let's consider the "complicated" example from the previous subsection. It was

$$B = \left[ \begin{array}{cccc|c} -1 & 2 & 1 & -5 & 6 \\ 0 & 0 & -1 & 1 & -5 \\ 1 & 3 & 3 & -1 & 0 \\ -1 & 5 & -1 & 0 & -1 \end{array} \right]$$

and we can enter that into Sage with

```
B = matrix( 4, 5, [-1, 2, 1, -5, 6, 0, 0, -1, 1, -5, 1, 3, 3,
                  -1, 0, -1, 5, -1, 0, -1 ] )
```

With the matrix  $B$  being built out of a very long list of numbers, we should be very careful that we've typed those correctly, omitting none and duplicating none. We check the correctness of  $B$  by typing `print B` alone on its own line, and see that we have entered that which we had intended to enter.

Before we continue, I'd like to share with you a useful and easy bit of notation. Sometimes we wish to refer to a position inside the matrix. For example, the 6 in the first row, fifth column of  $B$  can be written as  $B_{15}$ . Likewise, the 5 in the fourth row, second column of  $B$  can be written as  $B_{42}$ . Similarly, the two -5s are located at  $B_{14}$  and  $B_{25}$ . Remember, the row comes first, and the column comes second, just like RC Cola.

In Sage, the naming of the coordinates in matrix entries starts at 0 and not at 1. This means that the upper-left hand corner is `B[0][0]` in Sage, but  $B_{11}$  in mathematics. Similarly,  $B_{14}$  is typed in as `B[0][3]`, and  $B_{23}$  is typed in as `B[1][2]`. This anomaly is something that Sage inherited from the computer language Python, and it cannot be changed since Sage runs over Python.

By the way, an alternative way of entering a fairly large matrix is as follows:

```
B = matrix( [ [-1, 2, 1, -5, 6], [0, 0, -1, 1, -5],
              [1, 3, 3, -1, 0], [-1, 5, -1, 0, -1] ] )
```

The above code is a list of lists. Each list of five numbers enclosed in brackets and separated by commas represents one row; then there are four such lists, separated by commas and enclosed by brackets, to represent the entire matrix. Since this format gives away the number of rows and columns, we do not have to put the "4, 5" to inform Sage that  $B$  has 4 rows and 5

columns. You can feel free to use which ever format you might happen to feel more comfortable with.

Now that  $B$  has been entered (by either method) we should add the command `print B.rref()` on its own line, and we learn that the RREF is

$$\left[ \begin{array}{cccc|c} 1 & 0 & 0 & 0 & -107/7 \\ 0 & 1 & 0 & 0 & -12/7 \\ 0 & 0 & 1 & 0 & 54/7 \\ 0 & 0 & 0 & 1 & 19/7 \end{array} \right]$$

which gives the final answer of

$$w = -107/7 \quad x = -12/7 \quad y = 54/7 \quad z = 19/7$$

for the original system of equations. Now we must check our work. For example, we check the first equation with

```
print 2*(-12/7) - 5*(19/7) + 54/7
```

and

```
print 6 + -107/7
```

both of which come out to  $-65/7$ . The point is not that they come out to  $-65/7$ , but rather that they come out equal. Thus the first equation is satisfied. We check the other three equations similarly.

We really do have to plug each of the four values into each of the four equations, by the way. For example, the “imposter” solution

$$w = -139/7 \quad x = -8/7 \quad y = 64/7 \quad z = 29/7$$

satisfies the first three of those equations, but fails to satisfy the last one.

If you happened to read Section 1.5.2, you will recall that we did quite a lot of work to get  $B$ . Those many steps might have contained an error if we were sloppy. That’s another reason that we should check our work.

#### 1.5.4. Basic Summary

We’ve now learned how to use the `matrix` and `rref` commands to compute the RREF of a matrix, and therefore solve a linear system of equations. To do all the steps at once, I will often type

```
B = matrix( 4, 5, [-1, 2, 1, -5, 6, 0, 0, -1, 1, -5, 1, 3, 3,
                  -1, 0, -1, 5, -1, 0, -1 ] )
```

```
print "Question:"
print B
print "Answer:"
print B.rref()
```

The lines containing `print "Question:"` and `print "Answer:"` are entirely optional, but help make the output more readable. They also encourage me to check that I had actually entered the matrix which I had intended



to enter, without any typos. Somehow, I almost always make a typographical error, which I have to go back and correct. The output produced is below:

Question:

```
[-1  2  1 -5  6]
[ 0  0 -1  1 -5]
[ 1  3  3 -1  0]
[-1  5 -1  0 -1]
```

Answer:

```
[ 1  0  0  0  0 -107/7]
[ 0  1  0  0  0 -12/7]
[ 0  0  1  0  0  54/7]
[ 0  0  0  0  1  19/7]
```

### 1.5.5. The Identity Matrix

Let's look at these RREFs that we've computed so far. Do you see a pattern in them? Or I should say, a pattern in all columns but the last column?

$$\left[ \begin{array}{ccc|c} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{array} \right] \quad \text{and} \quad \left[ \begin{array}{cccc|c} 1 & 0 & 0 & 0 & -107/7 \\ 0 & 1 & 0 & 0 & -12/7 \\ 0 & 0 & 1 & 0 & 54/7 \\ 0 & 0 & 0 & 1 & 19/7 \end{array} \right]$$

We can see that all the columns (excepting the last column) have zeros everywhere, except a very noticeable diagonal of ones. This pattern (of zeros everywhere but ones in the  $A_{11}$ ,  $A_{22}$ ,  $A_{33}$ ,  $\dots$ , positions) is a pattern that occurs extremely often. Unsurprisingly, it has a name—mathematicians call this “the identity matrix.”

Consider when you are forced to show “ID,” for example when being pulled over for a speeding ticket or when trying to enter a bar. The ID quickly displays your vital facts, such as your name, your birthdate, your height, your gender, your address, and so forth. Likewise, the identity matrix displays the vital facts about a system of linear equations. You can read the values of the variables off, by simply reading the last column.

### 1.5.6. A Challenge to Practice by Yourself

Using Sage, solve the following linear system of equations. Check your answers with a hand calculator.

$$\begin{aligned} 3481x + 59y + z &= 0.87 \\ 6241x + 79y + z &= 0.61 \\ 9801x + 99y + z &= 0.42 \end{aligned}$$

Note: this problem might seem artificial, but it actually an applied problem which we will revisit shortly. Like many application-based problems, the answers have decimal points, which hopefully should not frighten you. We'll see the answer shortly.

### 1.5.7. Vandermonde's Matrix

In economics or in business, one often speaks a great deal about the price-demand curve. After all, as the price of a product goes up, then the demand for it should fall and as the price decreases, the demand should go up, at least under standard conditions.

Sometimes it would be useful to know the curve but it can usually only be computed by surveying potential customers. The standard technique is to find 2000 people, show each of them the product, and ask them if they would buy it or not. They get the product<sup>4</sup> for free in return for participating in the survey. The key is that of those 2000 people, perhaps 20 different prices would be used, each price being shown to 100 people.

That process will deliver 20 data points, which is often totally sufficient. However, sometimes it is better to have an actual function. Typically, a quadratic function is sought. There are good reasons for this, but it would be a digression<sup>5</sup> to explore them here. Imagine that a friend of yours with a startup company has done such a survey, but with 3 prices and 300 people, for some snazzy new product.

- At the price of \$ 59, the survey says that 87% would buy it.
- At the price of \$ 79, the survey says that 61% would buy it.
- At the price of \$ 99, the survey says that 42% would buy it.

Every quadratic function can be written in the form

$$f(x) = ax^2 + bx + c$$

but the question becomes: how can we find  $a$ ,  $b$ , or  $c$ ?

With this in mind, and assuming  $f(x)$  models the price-demand relationship well, then it really must be the case that  $f(59) = 0.87$ ,  $f(79) = 0.61$ , and  $f(99) = 0.42$ . With that in mind, then we could write the following equations

$$\begin{aligned} a(59^2) + b(59) + c &= 0.87 \\ a(79^2) + b(79) + c &= 0.61 \\ a(99^2) + b(99) + c &= 0.42 \end{aligned}$$

Surprisingly, these equations are linear, which is not necessarily expected since we are working with a quadratic function. All we must do is write the

---

<sup>4</sup>Unsurprisingly, this technique is more common in small consumer electronics or food items, and less common among luxury sports-car manufacturers.

<sup>5</sup>Let it be said that fitting high-degree polynomials to data points is entirely feasible, but extremely unwise, due to a phenomenon called "over fitting."

matrix down:

$$\left[ \begin{array}{ccc|c} 59^2 & 59 & 1 & 0.87 \\ 79^2 & 79 & 1 & 0.61 \\ 99^2 & 99 & 1 & 0.42 \end{array} \right]$$

and ask Sage to compute the RREF. However, you've already done that yourself! You already computed the solution

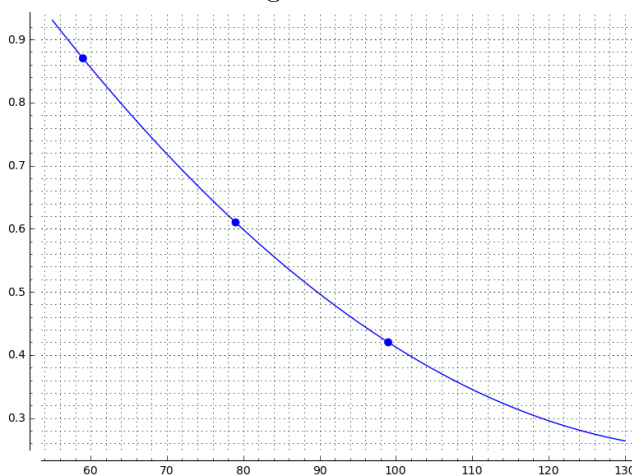
$$a = 0.0000875 \quad b = -0.025075 \quad c = 2.0448375$$

as practice, on Page 25 as Section 1.5.6.

Finally, we conclude that the function is

$$f(p) = 0.0000875p^2 - 0.025075p + 2.0448375$$

The graph of this function is given below:



If you are curious, the code which produced that plot was

$$f(x) = 0.0000875*x^2 - 0.025075*x + 2.0448375$$

```
plot( f, 55, 130, gridlines='minor') + point( [59, 0.87],
size=50 ) + point([79, 0.61], size=50) + point(
[99, 0.42], size=50 )
```

The general technique of fitting a degree- $n$  polynomial to  $n+1$  points using a matrix is due to Alexandre-Théophile Vandermonde (1735–1796), and therefore the matrices are called Vandermonde Matrices after him. For example, to fit a fourth-degree polynomial to five data points  $(x_1, y_1)$ ,  $(x_2, y_2)$ ,  $\dots$ ,  $(x_5, y_5)$ , we would use the matrix:

$$\left[ \begin{array}{cccc|c} (x_1)^4 & (x_1)^3 & (x_1)^2 & (x_1) & 1 & y_1 \\ (x_2)^4 & (x_2)^3 & (x_2)^2 & (x_2) & 1 & y_2 \\ (x_3)^4 & (x_3)^3 & (x_3)^2 & (x_3) & 1 & y_3 \\ (x_4)^4 & (x_4)^3 & (x_4)^2 & (x_4) & 1 & y_4 \\ (x_5)^4 & (x_5)^3 & (x_5)^2 & (x_5) & 1 & y_5 \end{array} \right]$$

### 1.5.8. The Semi-Rare Cases

Previously, our examples gave us one unique solution. Now we're going to examine the two semi-rare cases. These occur if there is one row of all zeros at the bottom of the RREF, ending in either a non-zero number, or ending in zero. The system of equations is

$$\begin{aligned}x + 2y + 3z &= 7 \\4x + 5y + 6z &= 16 \\7x + 8y + 9z &= 24\end{aligned}$$

That results in the matrix

$$C1 = \left[ \begin{array}{ccc|c} 1 & 2 & 3 & 7 \\ 4 & 5 & 6 & 16 \\ 7 & 8 & 9 & 24 \end{array} \right]$$

and that we shall enter into Sage with

```
C1 = matrix(3,4, [1, 2, 3, 7, 4, 5, 6, 16, 7, 8, 9, 24])
```

Next, to find the RREF we type

```
C1.rref( )
```

and receive back

$$\begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This translates into

$$\begin{aligned}x - z &= 0 \\y + 2z &= 0 \\0 &= 1\end{aligned}$$

The bottom equation says  $0 = 1$ , which is clearly not true! There is no way to satisfy the requirement that  $0 = 1$ . Therefore, this equation has no solutions. Instead of “the answer” being a solution, a list of solutions, or infinitely many solutions, the answer is the sentence “This linear system of equations has no solutions.” That will be the outcome whenever the RREF has a row with all zeros except the last column, which is a non-zero number. Whenever you see a row with all zeros, except in the last column where a number other than zero is found, then you must remember that the system has no solutions.

Another interesting case is to change the 24 in the last equation to 25. That results in the matrix

$$C2 = \left[ \begin{array}{ccc|c} 1 & 2 & 3 & 7 \\ 4 & 5 & 6 & 16 \\ 7 & 8 & 9 & 25 \end{array} \right]$$

and that we shall enter into Sage with

```
C2 = matrix(3,4, [1, 2, 3, 7, 4, 5, 6, 16, 7, 8, 9, 25])
```

Now, to find the “solution,” we type  
`C2.rref( )`  
 and receive back

```
[ 1  0  -1  -1 ]
[ 0  1   2   4 ]
[ 0  0   0   0 ]
```

As you can see, this RREF has a row of all zeros, ending in a zero. That usually indicates infinitely many solutions, a point we will clarify shortly. Some instructors allow you to write “infinitely many solutions” and in certain application problems you would not care to provide a way of saying what those solutions are.

However, some of the time, you’d like to know what the solutions are. This is especially true in geometry-related problems. In any case, you can’t make a list, because there infinitely many solutions. Often, even though the solution space is infinite, it turns out that it has what mathematicians call “one degree of freedom.” This means that there’s a free variable, which you can set to whatever value that you want. However, once you have done so, all the other variables are determined by this choice. That process is explained in Appendix D, along with many other facts about linear systems that have infinitely many solutions.

### When are there Infinitely Many Solutions?

So what does a row of zeros really tell us? Most of the time, your system of equations will have two equations in two unknowns; three equations in three unknowns; four equations in four unknowns; or five equations in five unknowns, and so forth. The technical term for this is an “exactly-defined” linear system. An exactly-defined linear system results in a  $2 \times 3$ ,  $3 \times 4$ ,  $4 \times 5$ ,  $5 \times 6$ , or  $n \times (n + 1)$  matrix. The “rule of thumb” that I’m about to present will work whenever there are an equal number of equations as unknowns, which means that the system is exactly defined, i.e. there is one more column than the number of rows.

Here’s the rule of thumb: If we see the tell-tale sign of “no solutions,” then we ignore all other information, and declare “no solutions.” Barring that, if there is a row of zeros, then there will be infinitely many solutions.

However, be warned! This rule does not work if there are “too few” or “too many” equations. If there are too many equations (too many rows), the system is said to be “over defined.” If there are too many variables (too many columns), the system is said to be “under defined.” For example, in Section D.3 on Page 314, I will list some counterexamples that demonstrate that this rule of thumb is false for matrices that are over defined or under defined. If you are curious, under-defined systems of linear equations come up in some cool analytic geometry questions, and over-defined systems of linear equations come up in cryptanalysis—and were used in my PhD dissertation.

Finally, it should be noted that in Appendix D, you can learn the relatively straight-forward technique that maps out exactly how to handle linear systems of equations with infinitely many solutions, regardless of the number of rows and columns. The trick there is only slightly more complicated, but it always works. Furthermore, it gives you formulas for producing as many solutions from the set of infinitely many solutions, as you might want to generate. You can also use those formulas for other purposes—for example, to test if a candidate solution is actually one of the infinitely many solutions.

Please do not imagine that linear systems with infinitely many solutions are terribly rare—they do occasionally come up in applications. However, they are uncommon enough that we will not burden you with those details yet, but we will wait until Appendix D.

## 1.6. Making your Own Functions in Sage

It is extremely easy to define your own functions<sup>6</sup> in Sage. To me, this is one of the most brilliant design features of Sage, that the creators made functions very easy to use. You can make your own function using exactly the symbols that you would normally use in writing the function down with your pencil on homework or writing on the whiteboard.

### Defining a Function:

For example, if you want to define  $f(x) = x^3 - x$  then you type

```
f(x) = x^3-x
f(2)
```

where we have defined  $f(x)$  and asked it the value of  $f(2)$ . You could instead ask for  $f(3)$ , or any other number. Likewise if you wanted to define  $g(x) = \sqrt{1-x^2}$  you would type

```
g(x) = sqrt( 1-x^2 )
g(1/4)
```

where we have defined  $g(x)$  and asked what  $g(1/4)$  evaluates to. You could just as easily ask for  $g(1/2)$  or any other number in the domain of  $g$ .

To evaluate several values, you can use the **print** command.

```
g(x) = sqrt( 1-x^2 )
print g(0.1)
print g(0.01)
print g(0.001)
print g(0.0001)
```

---

<sup>6</sup>Just to clarify, I am talking about mathematical functions, such as  $f(x) = x^3$ . However, if you are an experienced computer programmer, you might know about defining functions in a programming language—sometimes called procedures, methods, or subroutines. That will be discussed in Section 5.2 on Page 226.

```
print g(0.00001)
print g(0.000001)
```

If perhaps you happen to have started calculus, you can see that the above output would help you evaluate

$$\lim_{x \rightarrow 0^+} g(x) = 1$$

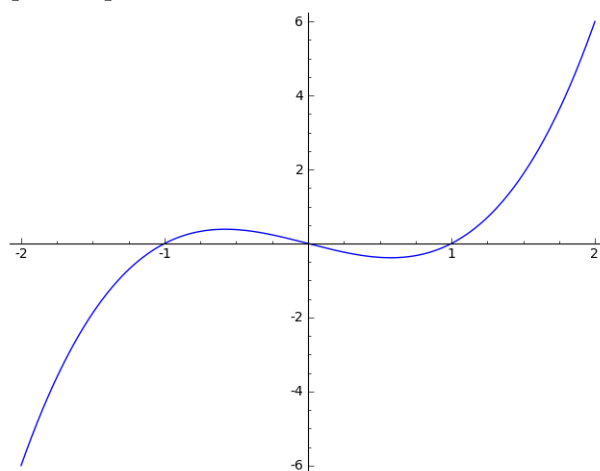
numerically, or help confirm your algebraic answer. If you haven't studied calculus, then you can ignore this remark!

### Plotting Functions:

Now that we've defined  $f(x)$  and  $g(x)$ , we can plot them with

```
f(x) = x^3-x
plot( f, -2, 2 )
```

you get the expected plot



Likewise if you type

```
g(x) = sqrt( 1-x^2 )
plot( g, 0, 1 )
```

which is equivalent to

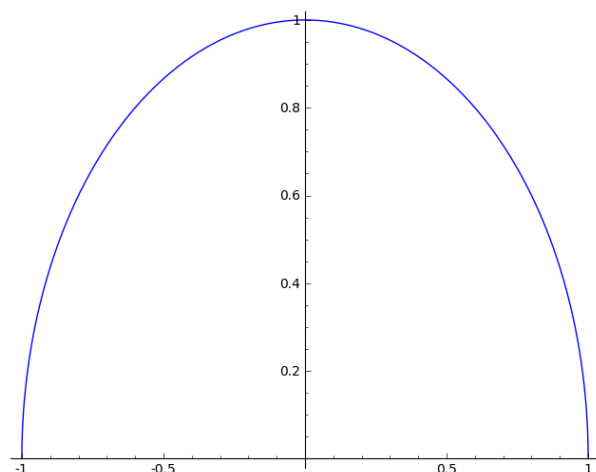
```
g(x) = sqrt( 1-x^2 )
plot( g(x), 0, 1 )
```

then you get the graph of  $g(x)$  on  $0 \leq x \leq 1$ .

Last but not least, if you type

```
g(x) = sqrt( 1-x^2 )
plot( g )
```

which is rather abbreviated—omitting the interval of the  $x$ -axis that you want—then Sage will assume that you want  $-1 \leq x \leq 1$ , by default. This produces the plot



Note that you really do have to keep the definitions of  $f(x)$  or  $g(x)$  in the window and written out each time you hit “Evaluate.” This is a feature of the Sage cell server. It has amnesia, and treats each press of the button “Evaluate” as its own separate mathematical universe. This is for the beginner’s benefit, because it makes the correcting of minor typographical errors very easy. You just fix the error, click “Evaluate” and all is well. SageMathCloud works just slightly differently, as will be explained on Page 301.

### Using Intermediate Variables:

Sometimes you’ll be using the same value a lot, and you want to store it somewhere. For example, if you type  $355/113$  a lot, then you can type

```
k=355/113
```

which silently stores the value  $355/113$  in the variable  $k$ . You can use it in any later line of the Sage cell-server window, such as

```
k=355/113
print 2+k
print 1+k
```

which displays the correct values of  $581/113$  and  $468/113$ . If you go back and change the value of  $k$  however, you have to make sure you use the “evaluate” button to re-evaluate your formulas.

Also, if you replace  $2+k$  with  $2.0+k$  then you get decimal approximations instead. If a mathematical statement is written entirely in terms of integers and rational numbers, Sage will provide an exact answer. If a decimal is introduced, it will provide a decimal approximation instead.

The intermediate variables technique is great for things in physics like the acceleration due to gravity, or the speed of light, which do not change during a problem.

As it turns out, this particular  $k$  is an approximation of  $\pi$ , kind of like  $22/7$  but much better. If we wanted to find the relative error of  $k$  as an



approximation of  $\pi$  we would remember the formula

$$\text{relative error} = \frac{\text{approximation} - \text{truth}}{\text{truth}}$$

and type

```
k=355/113
```

```
N( (k-pi)/pi )
```

and learn that the relative error is around 84.9 parts per billion. Not bad. This approximation was found by the Chinese Astronomer Zu Chongzhi, also known as Tsu Ch'ung-Chih. We can compare 355/113 to 22/7 with

```
N( (22/7-pi)/pi )
```

and learn that this much more common approximation of  $\pi$  as 22/7 has a relative error around 402.5 parts per million. That's much worse! In fact, 22/7 is 4740.9 times worse, so it is kind of a pity that we do not teach students 355/113, but instead we teach them 22/7.

### Composition of Functions:

You can also compose two functions very easily in Sage. Consider the following two functions

$$f(x) = 3x + 5 \quad \text{and} \quad g(x) = x^3 + 1$$

Perhaps we wish to explore  $f(g(x))$ . The easy way to do this is to define a third function,  $h(x) = f(g(x))$ . Then perhaps we might want to print that function, and the values of  $h(x)$  at  $x = 1$ ,  $x = 2$ , and  $x = 3$ . The Sage code for this is straight forward:

```
f(x) = 3*x + 5
```

```
g(x) = x^3 + 1
```

```
h(x) = f(g(x))
```

```
print h(x)
```

```
print h(1)
```

```
print h(2)
```

```
print h(3)
```

That produces the output

```
3*x^3 + 8
```

```
11
```

```
32
```

```
89
```

as desired. However, if we replace the third line with  $h(x) = g(f(x))$ , then we get completely different output

```
(3*x + 5)^3 + 1
```

```
513
```

```
1332
```

```
2745
```

Of course, there's no reason to expect  $f(g(x)) = g(f(x))$  in general. The way to see that in this specific case is to work out the two functions using algebra. We can compute

$$\begin{aligned} f(g(x)) &= 3(g(x)) + 5 \\ &= 3(x^3 + 1) + 5 \\ &= 3x^3 + 3 + 5 \\ &= 3x^3 + 8 \end{aligned}$$

which is completely different than

$$\begin{aligned} g(f(x)) &= g(3x + 5) \\ &= (3x + 5)^3 + 1 \\ &= 27x^3 + 135x^2 + 225x + 125 + 1 \\ &= 27x^3 + 135x^2 + 225x + 126 \end{aligned}$$

### Multivariate Functions:

This section has dealt with functions of one variable. Sage can very easily and simply handle functions of multiple variables. For example,

$$f(x, y) = (x^4 - 5x^2 + 4 + y^2)^2$$

We'll cover this in detail in Section 4.1 on Page 127.

## 1.7. Using Sage to Manipulate Polynomials

Now we're going to explore some uses of functional notation, from the previous section, to polynomials. First, let's see that Sage can add polynomials very easily. We're going to explore the following polynomials.

$$\begin{aligned} a(x) &= x^2 - 5x + 6 \\ b(x) &= x^2 - 8x + 15 \end{aligned}$$

If we type the following code

```
a(x) = x^2 - 5*x + 6
b(x) = x^2 - 8*x + 15
```

```
a(x) + b(x)
```

we get the correct answer of  $2x^2 - 13x + 21$ . Note, it is very important to remember the asterisk between the 5 and the  $x$  as well as the 8 and the  $x$ .

Similarly, if you change the  $+$  sign between  $a(x)$  and  $b(x)$  into a  $-$  sign, then we also get the correct answer of  $3x - 9$ . As you can imagine, the product can be conveniently calculated as well:

```
a(x) = x^2 - 5*x + 6
b(x) = x^2 - 8*x + 15
```

```
a(x)*b(x)
```

but this provides the unfinished yet undeniably true answer

```
(x^2 - 5*x + 6)*(x^2 - 8*x + 15)
```

Instead, we can do the following

```
a(x) = x^2 - 5*x + 6
```

```
b(x) = x^2 - 8*x + 15
```

```
g(x) = a(x)*b(x)
```

```
g.expand()
```

and we get  $g(x)$  written without parentheses. In computer algebra, that's called "expanded form." The expanded form of our  $g(x)$  happens to be

```
x |--> x^4 - 13*x^3 + 61*x^2 - 123*x + 90
```

The symbol `|-->` means "evaluates to." What Sage is telling you here is that  $g(x)$  is a function that sends  $x$  to  $x^4 - 13x^3 + 61x^2 - 123x + 90$ .

Instead, we can write `g.factor()` in place of `g.expand()` in which case we get the following output:

```
(x - 2)*(x - 3)^2*(x - 5)
```

You can also factor more directly with

```
a(x) = x^2 - 5*x + 6
```

```
factor( a(x) )
```

and get the correct answer of  $(x - 2)(x - 3)$ . Alternatively we can be even more direct by typing

```
factor( x^2 - 8*x + 15 )
```

to obtain  $(x - 3)(x - 5)$ .

You can also compute the "greatest common divisor" or `gcd` of two polynomials. For example, the following code

```
a(x) = x^2 - 5*x + 6
```

```
b(x) = x^2 - 8*x + 15
```

```
gcd( a(x), b(x) )
```

provides the correct answer of  $(x - 3)$ . Of course, we knew that from the factorizations that we found earlier. The common factor of  $(x - 2)(x - 3)$  and  $(x - 3)(x - 5)$  is clearly  $(x - 3)$ .

It should be noted that these examples were trivial because we used quadratic polynomials. The following polynomial is one that I would not want to factor by hand—though, admittedly, the integer roots theorem<sup>7</sup>

---

<sup>7</sup>If  $f(x)$  is a polynomial with all integer coefficients, then any integer root must be a divisor of the constant term of  $f(x)$ . Most numbers have a modest number of divisors, so it is relatively easy to simply check each of them, and thereby have a complete list of all the integer roots of  $f(x)$ . Note that this works even for quintic and higher degree polynomials, which are otherwise extremely difficult to work with. There is also a "rational roots theorem," but we will not discuss that here.

```

a(x) = x^2 - 5*x + 6
b(x) = x^2 - 8*x + 15

f(x) = a(b(x))

print "Direct:"
print f(x)
print "Expanded:"
print f.expand()
print "Factored:"
print f.factor()

```

FIGURE 1. Some Sage Code to Explore Composing Two Functions

would produce the correct answer using only a pencil and a great deal of human patience

```
factor( x^4 - 60*x^3 + 1330*x^2 - 12900*x + 46189 )
```

Speaking of the integer roots theorem, we could type `factor(46189)` and get `11 * 13 * 17 * 19` to help us find roots of that quartic polynomial. The `factor` command deals with both factoring a polynomial and factoring an integer. The command `divisors(46189)`, as an alternative, lists all the positive integers which divide 46,189. Naturally, that's a longer list than `factor`, which only would list the prime divisors (and their exponents, if any). Compare `factor(2048)` and `divisors(2048)`, if you like.

The composition of functions is also not a problem. The code given in Figure 1 will compose two polynomials and produce the answer in three forms. It is noteworthy that this is our first real “program” in Sage, in the sense that it is the first time that we've used more than two or three commands at once.

In particular, our program produces the output:

```

Direct:
(x^2 - 8*x + 15)^2 - 5*x^2 + 40*x - 69
Expanded:
x |--> x^4 - 16*x^3 + 89*x^2 - 200*x + 156
Factored:
(x^2 - 8*x + 13)*(x - 2)*(x - 6)

```

The above code demonstrates how it can be very useful to label parts of your output with `print` commands if you're outputting more than one or two items. Such labeling (sometimes called “annotating the output”) is a good general practice as you learn to tackle more and more complex tasks in Sage.

Just as we saw at the end of Section 1.6, you get a different answer if you change `f(x) = a(b(x))` into `f(x) = b(a(x))`. Go ahead, give it a try, and see for yourself!

### 1.8. Using Sage to Solve Problems Symbolically

When we say that a computer has solved a problem for us, that can come out to be in one of two flavors: symbolically, or numerically. When we solve numerically, we get a decimal expansion for a (very good) approximation of the answer. When we solve symbolically, we get an exact answer, often in terms of radicals or other complicated functions. It is a matter of saying if the solutions to

$$\frac{x^2}{2} - x - 2 = 0$$

are  $1 + \sqrt{5}$  and  $1 - \sqrt{5}$  or saying that they are

$$-1.23606797749979 \text{ and } 3.23606797749979$$

The former is an example of a symbolic solution, and the latter a numerical solution. Often, a numerical answer is desired. However, if the answer is a formula, then a symbolic solution is the only way to go. This section is about symbolic solution, and then the next section is about numerical solution.

#### 1.8.1. Solving Single-Variable Formulas

First, let's try solving some single-variable problems. We can type

```
solve(x^2 + 3*x+2, x)
```

and then we obtain

```
[x == -2, x == -1]
```

It is important to remember the asterisk! To be precise, to type

```
solve(x^2 + 3x+2, x)
```

would be wrong, because of the absence of the asterisk between the “3” and the “x.” Another, more illustrative example of a symbolic computation is

```
solve(x^2+9*x+15==0,x)
```

which outputs

```
[x == -1/2*sqrt(21) - 9/2, x == 1/2*sqrt(21) - 9/2]
```

By the way, do you notice how the solutions are in a comma-separated list, enclosed in square brackets? That's an example of a list, in notation that Sage inherited from the computer language Python.

One way to really see what symbolic versus exact is about is to compare

```
3+1/(7+1/(15+1/(1+1/(292+1/(1+1/(1+1/6))))))
```

which returns  $1,354,394 / 431,117$  to

```
N( 3+1/(7+1/(15+1/(1+1/(292+1/(1+1/(1+1/6)))))) )
```

which returns  $3.14159265350241$ . That could easily be mistaken for  $\pi$ . In fact it is really close to  $\pi$  with relative error  $2.78 \times 10^{-11}$ .

There is no restriction to polynomials. You can also type

```
var('theta')
solve(sin(theta)==1/2, theta)
to get
```

```
[theta == 1/6*pi]
```

but note that there are many values for which  $\sin \theta = 1/2$ . For example,

$$\theta \in \left\{ \dots, -\frac{31\pi}{6}, -\frac{23\pi}{6}, -\frac{19\pi}{6}, -\frac{11\pi}{6}, -\frac{7\pi}{6}, \frac{\pi}{6}, \frac{5\pi}{6}, \frac{13\pi}{6}, \frac{17\pi}{6}, \frac{25\pi}{6}, \frac{29\pi}{6}, \dots \right\}$$

are several values of  $\theta$  that solve  $\sin \theta = 1/2$ .

### Declaring Variables:

You might be confused to see that `var` command above. The idea is that you need to tell Sage that this variable is not yet known. We’ve used variables before, but that’s because we had assigned values to them. When you want a variable to represent some unknown quantity, you must use `var`. The variable  $x$  is always pre-declared, you do not need to declare it—Sage assumes that  $x$  is an unknown.

#### 1.8.2. Solving Multivariable Formulas

Now we’re going to use Sage to re-derive the quadratic formula. First we must declare our variables with:

```
var('a b c')
and then we type
solve( a*x^2 + b*x + c == 0, x )
and receive back
[x == -1/2*(b + sqrt(-4*a*c + b^2))/a,
x == -1/2*(b - sqrt(-4*a*c + b^2))/a]
```

which has some terms in an unusual order but is undoubtedly correct. Once again, the square brackets and comma indicate a list in Sage, something which Sage inherited from the computer language Python.

Of course, you probably know the quadratic formula very well. (At least I hope you do!) However, you probably do not know Cardano’s Cubic Formula. In Section 1.8.5 on Page 43, we’ll see how Sage can “rediscover” Cardano’s formula on the spot when similarly challenged.

### Declaring Several Variables at Once:

By the way

```
var('a b c')
```

is merely an abbreviation for

```
var('a')
var('b')
var('c')
```

Also note that the following for forms are equivalent  
`var("a, b, c")`    `var("a b c")`    `var('a b c')`    `var('a, b, c')`  
 and you can use which ever one you prefer.

### 1.8.3. Linear Systems of Equations

You can solve several equations simultaneously, whether they are linear or non-linear. An easy case might be to solve

$$\begin{aligned}x + b &= 6 \\x - b &= 4\end{aligned}$$

which would be done by typing

```
var('b')
solve( [x+b == 6, x-b == 4], x, b )
```

Again, note the use of square brackets and commas to form a list in Sage. You can enclose any data with [ and ], and separate the entries with commas, to make a list.

Also, it is important to point out that there is no harm in “declaring”  $b$  twice. Two `var` commands do no harm to each other. On the other hand, there is also no need to “declare”  $b$  twice, as once it is declared, Sage will remember that it is a variable.

A more typical linear system might be

$$\begin{aligned}9a + 3b + 1c &= 32 \\4a + 2b + 1c &= 15 \\1a + 1b + 1c &= 6\end{aligned}$$

and to solve that we’d type

```
var('a, b, c')
solve( [9*a + 3*b + c == 32, 4*a + 2*b + c == 15,
a + b + c == 6], a, b, c )
```

and Sage gives the answer

```
[[a == 4, b == -3, c == 5]]
```

which is correct. Of course, this is exactly the linear system of equations that you would use if someone asked you to find the parabola connecting the points (3, 32) and (2, 15) as well as (1, 6). That would be  $f(x) = 4x^2 - 3x + 5$ . This is another example of a Vandermonde Matrix (except that we didn’t use a matrix here), as we saw defined in Section 1.5.7 on Page 26.

Naturally, linear systems of equations can also be solved with matrices. In fact, Sage is quite useful when working with matrices, and can remove much of the tedium normally associated with matrix algebra. That was discussed in the matrix section (Section 1.5) on Page 19.

### 1.8.4. Non-Linear Systems of Equations

While linear equations are very easy to solve via matrices, the non-linear case is usually much harder. First, we will warm up with just one equation, but a highly-non-linear one. Let us try to solve

$$x^6 - 21x^5 + 175x^4 - 735x^3 + 1624x^2 - 1764x + 720 = 0$$

which can be done by

```
solve( x^6 - 21*x^5 + 175*x^4 - 735*x^3 + 1624*x^2 - 1764*x +
720 == 0, x)
```

which gives

```
[x == 5, x == 6, x == 4, x == 2, x == 3, x == 1]
```

That is a list of six answers, because that degree six polynomial has six roots. It was easy to read this time, but you can also access each answer one at a time. Instead, we type

```
answer = solve( x^6 - 21*x^5 + 175*x^4 - 735*x^3 + 1624*x^2 -
1764*x + 720 == 0, x)
```

and then we can type `print answer[0]` or `print answer[1]` to get the first or second entries. To get the fifth entry of that list, we'd type `print answer[4]`. This is because Sage is built out of Python, and Python numbers its lists from 0 and not from 1. There are reasons for this, based on some very old computer languages.

Now consider this problem, suggested by Prof. Jason Grout, of Drake University. To solve

$$\begin{aligned} p + q &= 9 \\ qy + px &= -6 \\ qy^2 + px^2 &= 24 \\ p &= 1 \end{aligned}$$

we would type

```
var('p q y')
eq1 = p+q == 9
eq2 = q*y + p*x == -6
eq3 = q*y^2 + p*x^2 == 24
eq4 = p == 1
solve( [eq1, eq2, eq3, eq4 ], p, q, x, y )
```

which produces

```
[[p== 1, q== 8, x == -4/3*sqrt(10) - 2/3, y ==
1/6*sqrt(2)*sqrt(5) -2/3], [p== 1, q== 8, x == 4/3*sqrt(10)
- 2/3, y == -1/6*sqrt(2)*sqrt(5) - 2/3]]
```

As you can see, that is a list of lists, again using square brackets and commas. Clearly, that is a very hard to read mess. Using the technique that we learned when analyzing the degree six polynomial above, we replace the last line with



```
answer=solve( [eq1, eq2, eq3, eq4 ], p, q, x, y )
```

Now we can type

```
print answer[0]
print answer[1]
```

and we get

```
[p==1, q==8, x== -4/3*sqrt(10) - 2/3, y== 1/6*sqrt(2)*sqrt(5) - 2/3]
[p==1, q==8, x== 4/3*sqrt(10) - 2/3, y== -1/6*sqrt(2)*sqrt(5) - 2/3]
```

This is Sage's way of telling you:

Solution #1:  $p = 1, q = 8, x = -\frac{4}{3}\sqrt{10} - \frac{2}{3}, y = \frac{\sqrt{10}}{6} - \frac{2}{3}$

Solution #2:  $p = 1, q = 8, x = \frac{4}{3}\sqrt{10} - \frac{2}{3}, y = -\frac{\sqrt{10}}{6} - \frac{2}{3}$

Since there are only two answers, we cannot ask for a third. In fact, if we type `print answer[2]` then we get

Traceback (click to the left of this block for traceback)

...

`IndexError: list index out of range`

which is Sage's way of telling you that it has already given you all the answers for that problem—the list does not contain a 3rd element.

Now let's try to intersect the hyperbola  $x^2 - y^2 = 1$  with the ellipse  $x^2/4 + y^2/3 = 1$ . We type

```
var('y')
solve([x^2-y^2==1, (x^2)/4+(y^2)/3==1], x, y)
```

and obtain

```
[x == -4/7*sqrt(7), y == -3/7*sqrt(7)], [x == -4/7*sqrt(7),
y == 3/7*sqrt(7)], [x == 4/7*sqrt(7), y == -3/7*sqrt(7)],
[x == 4/7*sqrt(7), y == 3/7*sqrt(7)]
```

which is unreadable. Once again, we change the command to be

```
answer=solve([x^2-y^2==1, (x^2)/4+(y^2)/3==1], x, y)
```

and then type

```
print answer[0]
print answer[1]
print answer[2]
print answer[3]
print answer[4]
```

and that produces four answers and an error message

```
[x == -4/7*sqrt(7), y == -3/7*sqrt(7)]
[x == -4/7*sqrt(7), y == 3/7*sqrt(7)]
[x == 4/7*sqrt(7), y == -3/7*sqrt(7)]
[x == 4/7*sqrt(7), y == 3/7*sqrt(7)]
```

Traceback (click to the left of this block for traceback)

...

`IndexError: list index out of range`  
 which is Sage’s way of telling you

$$\text{Solution \#1: } x = \frac{4}{7}\sqrt{7}, y = \frac{3}{7}\sqrt{7}$$

$$\text{Solution \#2: } x = \frac{4}{7}\sqrt{7}, y = -\frac{3}{7}\sqrt{7}$$

$$\text{Solution \#3: } x = -\frac{4}{7}\sqrt{7}, y = \frac{3}{7}\sqrt{7}$$

$$\text{Solution \#4: } x = -\frac{4}{7}\sqrt{7}, y = -\frac{3}{7}\sqrt{7}$$

but that there is no “Solution 5.”

Because finding the roots of a polynomial (or finding the solutions to an equation) is a common mathematical task, there are some other ways to display the solutions. For example, there’s some code which will display each root on its own line, and you can choose to show only real roots, only rational roots, or only integer roots. I will describe that in more detail in Section 5.7.10 on Page 275.

Furthermore, we aren’t limited to polynomials. We can type  
`solve( log( x^2 ) == 5/3, x )`

and we receive back

$$[x == -e^{(5/6)}, x == e^{(5/6)}]$$

both of which satisfy the given equation. Note that `log` in Sage means the natural logarithm, as was explained on 5.

We can also do

$$\text{solve(sin(x+y)==0.5,x)}$$

and obtain

$$[x == 1/6\pi - y]$$

but that’s clearly not comprehensive. For example, if  $x = 5\pi/6 - y$  then we have

$$\begin{aligned} \sin(x+y) &= \sin\left(\frac{5\pi}{6} - y + y\right) \\ &= \sin\left(\frac{5\pi}{6}\right) \\ &= 1/2 \end{aligned}$$

### Complex Numbers:

Normally, we expect an 8th degree polynomial to have 8 roots, over the complex plane, unless some are repeated roots. Therefore, if we ask “what are the roots of  $x^8 + 1$ ?” then surely we expect 8 roots. These are the 8 eighth-roots of  $-1$  in the set of complex numbers. We can find them with

$$\text{solve}(x^8 == -1, x)$$

which produces

```
[x == (1/2*I + 1/2)*(-1)^(1/8)*sqrt(2), x == I*(-1)^(1/8), x ==
(1/2*I - 1/2)*(-1)^(1/8)*sqrt(2), x == -(-1)^(1/8), x == -(1/2*I +
1/2)*(-1)^(1/8)*sqrt(2), x == -I*(-1)^(1/8), x == -(1/2*I -
1/2)*(-1)^(1/8)*sqrt(2), x == (-1)^(1/8)]
```

giving us eight distinct complex numbers, all of which have  $-1$  as their eighth power. This can be calculated (by hand) more slowly with DeMoivre's formula, but for Sage, this is an easy problem. I simply copy-and-paste the first root and raise it to the eighth power, as below

```
( (1/2*I + 1/2)*(-1)^(1/8)*sqrt(2) )^8
```

and Sage returns the answer  $-1$ .

### 1.8.5. Advanced Cases

To see why we don't ask undergraduates to memorize Cardano's cubic formula, try typing in:

```
solve(x^3 + b*x + c==0, x)
```

That gives you Cardano's formula for a monic depressed cubic. A cubic polynomial is said to be depressed if the quadratic coefficient is zero, and monic means that the leading coefficient (here, the cubic coefficient) is one. The full version of the cubic formula would be given by:

```
var('d')
solve(a*x^3 + b*x^2 + c*x + d == 0, x)
```

which is just insanely complicated. Try it, and you'll see. In fact, it is so complicated that one would have to confess that it is not useable.

An even uglier formula would be the general formula for a quartic, or degree four, polynomial. Surely any quartic can be written

$$a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0 = 0$$

so we can ask Sage to solve it with

```
var('a0 a1 a2 a3 a4')
solve( a4*x^4 + a3*x^3 + a2*x^2 + a1*x + a0 == 0, x)
```

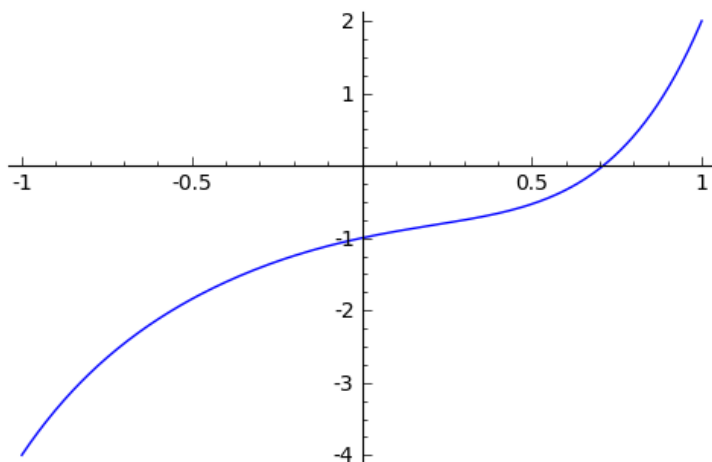
and we get an answer that is a formula of horrific proportions.

## 1.9. Using Sage as a Numerical Solver

The set of  $x$ -values that make  $f(x) = 0$  are commonly called "the roots" of  $f(x)$ . Let's suppose you need to know the value of a root of

$$f(x) = x^5 + x^4 + x^3 - x^2 + x - 1$$

a polynomial whose graph is given below.



Furthermore, suppose you are interested in a root between  $-1$  and  $1$ . This could come about because you graphed it (with or without Sage), and saw that there is a root in that region; or it could come about because you saw that  $f(-1) = -4$  but  $f(1) = 2$ , thus  $f$  clearly crosses the  $x$ -axis at some point between  $-1$  and  $1$ , though we don't know where just yet. Perhaps a problem in a textbook simply tells you that the root is between  $-1$  and  $1$ .

To solve for that root, you need only type this

```
find_root(x^5+x^4+x^3-x^2+x-1,-1,1)
```

and Sage tells you that

0.71043425578721398

is the root. This is a numerical approximation, because quintic polynomials have a very special property—if you don't know what the special property is, just ask any math teacher.

While it is an approximation, because the computer does several billion instructions per second, Sage invests computation time in refining the approximation. Therefore it is an approximation that is trustworthy to around an accuracy of  $10^{-16}$ , which is one tenth of a quadrillionth. This is a good approximation, by any standard.

Meanwhile, it turns out there's no root between  $-1$  and  $0$ . Perhaps you are told that, or perhaps you graph it (with or without Sage). If you type

```
find_root(x^5+x^4+x^3-x^2+x-1,-1,0)
```

then Sage tells you

```
RuntimeError: f appears to have no zero on the interval
```

which is perfectly honest, because there's no root in the interval for Sage to go and find! If you have no idea where the root is, you can type

```
find_root(x^5+x^4+x^3-x^2+x-1, -10^12, 10^12)
```

which is asking Sage to find a root that is between plus/minus one trillion.

Extremely sophisticated problems can be asked about. Here's a favorite: if someone asks you about  $x^x = 5$ , then you could try to find where  $x^x - 5 = 0$ . You would type

```
find_root(x^x-5, 1, 10)
```

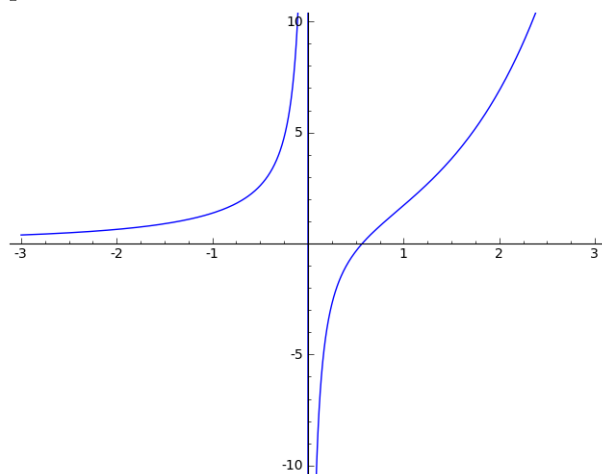
or equivalently

```
find_root(x^x==5, 1, 10)
```

Another one is to find where  $e^x = 1/x$  and you'd do that by finding a root of  $e^x - 1/x = 0$ . You could graph that with

```
plot( e^x - 1/x, -3, 3, ymin=-10, ymax=10)
```

obtaining the plot

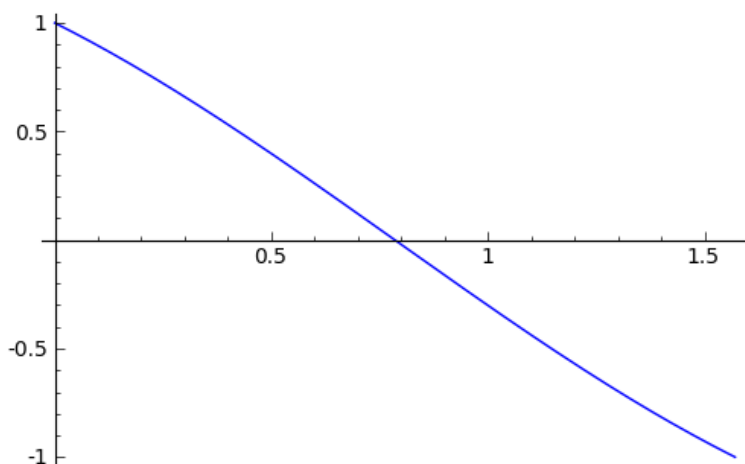


and then you can see the root is between  $-1$  and  $1$ . This means that we should type

```
find_root(e^x-1/x,-1, 1)
```

will result in finding out that the root is at  $x = 0.56714329040979539$ .

Last but not least, another favorite is to find where  $\sin x = \cos x$ . It happens to be the case that there's such an  $x$  between  $0$  and  $\pi/2$ , which you can see by graphing. The graph is



Now here you could ask for a root of  $(\sin x) - (\cos x)$ . Alternatively, you can also type

```
find_root( cos(x) == sin(x), 0, pi/2 )
```

which is just easier notation.

### Returning to an Old Problem about Compound Interest:

Earlier, on Page 6, we saw a problem involving compound interest, and finding how many months will be required to turn \$ 5000 into \$ 7000. Now let's see how to solve that with Sage in only one line, rather than the previous approach which required the human to do some algebra.

The first line of our analysis was

$$7000 = 5000(1 + 0.045/12)^n$$

and translating that into the language of Sage results in

```
7000 == 5000*(1+0.045/12)^n
```

then we should type

```
var('n')
find_root( 7000 == 5000*(1+0.045/12)^n, 0, 10000)
```

Note that the second line means that I'm searching for a value of  $n$  that makes the equation satisfied, and that the value of  $n$  should be between 0 and 10,000. This seems reasonable, because 10,000 months is a little over 833 years—surely it won't take that long. Note that it is vital to not include a comma inside of 10,000, as Sage's syntax forbids that use of the comma, as we saw on Page 2 and Page 3. The first command above (`var`) is just declaring  $n$  as a variable, which we saw on Page 38. In any case, we get the answer 89.894060933080027 as before, and learn that 90 months will be required.

## 1.10. Getting Help when You Need It

By now we have learned more than a few commands and it is important to discuss how you can use the built-in “help features” for Sage. There are several, each intended for a different purpose.

The one that I use most commonly is called tab-completion (first described on Page 4), and is meant for when you do not remember the exact name of a command. For example, perhaps I am confused if the correct command is `find_root`, `findroot`, `find_roots`, `root_find`, or maybe something else entirely! I would type `find` and then press the tab key on my keyboard (without clicking “Evaluate.”) I would then get a list of choices, most of which have nothing to do with my intentions. However, one is the command that I seek, `find_root`. Alternatively, if I had reached `find_r` before pressing the tab key, then there is only one possible command, and therefore Sage just completes the command for me.

Now let’s imagine that I have remembered the name of the command—perhaps `solve`—but not the exact syntax. Then what I should type is `solve?` with no space between the question mark and the command, and then press tab. A handy window pops up that will display lots of great information about the command. Feel free to read the discussion if you like, but I find it faster to merely scroll toward the examples and see how things are done there, mimicking what I find there as needed. By the way, the huge collection of information found in that window is called “the docstring,” which is short for “the documentation string.”

There is another way to use the `?` operator. If you put two question marks after a command name, such as `N??` then you will see the entire source code (usually written in the computer language Python) for the command. In this case you’ll see the Python code for the `N()` command which turns exact algebraic expressions into floating point numbers. Of course, the code is very advanced and would require much more knowledge of Python than this book will attempt to impart. (We will barely scratch the surface in Chapter Five).

Nonetheless, it is important that this feature (of displaying source code) is there for philosophical reasons. Often we have to know *how* a mathematical program will accomplish its task (in other words, how it will work internally) in order to figure out how to best format and preprocess our data prior to using that program. This comes up in all sorts of situations, and having access to the source code is essential. It also allows the ideas in the mathematical program to be used by other programmers in their own programs. In this way, the open-source community lives as a sort of “hive mind” sharing ideas widely. Also, mathematical software will inevitably have bugs. A colleague of mine found a bug in Mathematica, and notified the company via email about it. They acknowledged the bug, but three years later, it remains unfixed. In Sage, you can simply fix the bug yourself and submit

the change for peer review. If the change is found to be helpful and valid, it will be made. Sometimes the entire process can take as little as six weeks.

This now brings up an important point: you have to know the name of the command to use either of those first three methods of getting help. What if you cannot remember the name of the command? This happens often, and there are several solutions.

The official *Sage Reference Manual* can be found at <http://www.sagemath.org/doc/reference/> but it is not for beginners! Be warned, the manual is literally thousands of pages long. The manual is like an encyclopedia: you don't read it cover to cover, instead you look up what you need. Luckily, there is a handy "Quick Search" box on the left-hand side.

Tutorials are much more useful for most readers. Some are organized around a mathematical topic and others are organized around a target audience. There is a collection of Sage tutorials, a partial list of which can be found in Appendix C on Page 305.

There is also a way to search all the docstrings of all the commands in Sage. However, this will usually produce a very large number of results. Type, for example:

```
search_doc("laplace")
```

and you will see a huge response. It is not exactly human-readable. Here is a particular line of that output:

```
html/en/constructions/calculus.html:231:<div class="section"
      id="laplace-transforms">
```

The entire line refers to a webpage. The middle number, between the columns, is a line number—one is the top line, two is the second line, and so forth. The information after the second column is written in the HTML language. If you know HTML, then you can<sup>8</sup> use that information.

So far `search_doc` has not told us anything useful, but the information to the left of the first colon is very useful. That is a filename inside of a directory hierarchy that contains the documentation of Sage. Usually Sage is used through a server, such as the Sage cell server or SageMathCloud. However, one can do a local install, which means you are running Sage contained entirely inside your own computer, and not communicating with a remote server. This is not recommended except for experts.

What, then, should the rest of us (who use Sage through a web browser or SageMathCloud) do in order to get at the information? We must go to the following URL, which is essentially the information which `search_doc` provided, but in a different word order.

```
http://www.sagemath.org/doc/constructions/calculus.html?
```

As you can see, we replaced `html/en/` with

---

<sup>8</sup>If you do not know HTML, perhaps you might want to consider learning it. The HTML language is extremely useful and easy. Being able to make your own web-pages is an excellent skill, and highly marketable.



<http://www.sagemath.org/doc/>

in order to construct that URL. That substitution will work in general. By the way, the **en** refers to the English language, with **fr** referring to French, **de** to German, and **ru** to Russian, according to the normal two-letter language codes in use on the internet.

### 1.11. Using Sage to Take Derivatives

The command for the derivative is just **diff**. For example, to take the derivative of  $f(x) = x^3 - x$  you would type:

```
diff( x^3 - x, x)
```

to learn the answer

```
3*x^2 - 1
```

Or for  $f(x) = \sin(x^2)$  you would type:

```
diff( sin(x^2), x )
```

to receive the answer

```
2*x*cos(x^2)
```

Sometimes you might have earlier defined your own function by

```
g(x)=e^(-10*x)
```

earlier, so you can then type

```
diff(g(x), x)
```

to get the derivative, which is naturally

```
-10*e^(-10*x)
```

but you can also type

```
gprime(x) = diff(g, x)
```

which is silent, but then you can type **gprime(2)** or **gprime(3)** and it does what you expect. It tells you the values of  $g'(2)$  and  $g'(3)$ . You can also type

```
g(x)=e^(-10*x)
```

```
g.derivative()
```

We cannot, however, define a function called  $g'(x)$ . That's because Sage is built on top of the already existing computer language called Python, and the apostrophe has a predefined meaning in Python. This is unfortunate, but we can always write **gprime(x)** or **g\_prime(x)**, so it will not be a barrier for us.

Sage can also do very difficult derivatives:

```
diff( x^x, x)
```

which provides the answer

```
(log(x) + 1)*x^x
```

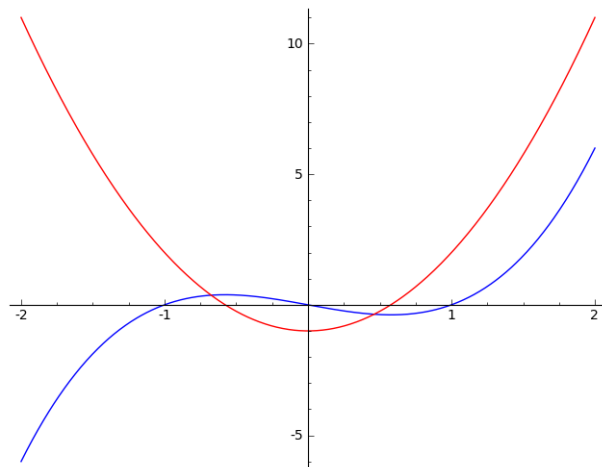
Now if you're interested in giving your calculus skills a workout, but possibly a very intense one, you can try to see how you would calculate the derivative of  $x^x$  with your pencil. By the way, it is important to note that `log` refers to the natural logarithm, not the common logarithm, as was explained on Page 5.

### 1.11.1. Plotting $f(x)$ and $f'(x)$ Together

One of my favorite bits of code in Sage is

```
f(x)=x^3-x
fprime(x)=f(x).derivative()
plot( f(x),-2,2, color='blue') + plot( fprime(x),-2,2, color='red')
```

and I use this quite frequently. This plots  $f(x)$  and  $f'(x)$  on the same graph, with  $f(x)$  in blue, and  $f'(x)$  in red. You can very easily change what function you are analyzing by changing that first line. Here we are analyzing  $f(x) = x^3 - x$ , in blue, and its derivative,  $f'(x) = 3x^2 - 1$ , in red. The plot is as follows:



### 1.11.2. Higher-Order Derivatives

While you can find the second derivative by using `diff` twice, you can jump directly to the second derivative with

```
diff( x^3-x, x, 2)
```

to get the answer

```
6*x
```

Third or higher derivatives are also no problem. For example, the third derivative can be found with

```
diff( x^3-x, x, 3)
```

For those who don't mind typing more, you can do

```
derivative( x^3-x, x, 2)
```

which is more readable than writing `diff`.

## 1.12. Using Sage to Calculate Integrals

The only thing you need to understand in order to do integrals in Sage is that there are really three types of integrals that can be done: a numerical approximate integral, an exact definite integral, and an indefinite integral. The first two types result in numbers, but they are found approximately or exactly, respectively. The third one is the symbolic antiderivative, which means that the answer is a function—in fact, the answer is a function whose derivative is what you had typed in. If this is confusing, then let us consider a simple example:

### An Example of Integration:

Using an integral, how would you calculate the area between the  $x$ -axis and  $f(x) = x^2 + 1$  on the interval  $3 \leq x \leq 6$ ? Well, that integral would be

$$\int_3^6 (x^2 + 1) \, dx = \left( \frac{1}{3}x^3 + x \right) \Big|_3^6 = \left( \frac{1}{3}6^3 + 6 \right) - \left( \frac{1}{3}3^3 + 3 \right) = 72 + 6 - (9 + 3) = 66$$

and that is a number. Here we've done it analytically, using calculus. This is an exact definite integral. It is definite, because it ends with a number. It is exact, because we did no approximations.

On the other hand,

$$\int (x^2 + 1) \, dx = \frac{1}{3}x^3 + x + C$$

is an indefinite integral. It produces a function, and not a number. Of course, we often calculate an indefinite integral on the way to calculating a definite integral; nonetheless, the definite and indefinite integrals are two distinct categories. If you want a function, you should use the indefinite integral, and if you want a number, then you should use the definite integral.

### The Indefinite Integral:

The indefinite integral is the simplest. For example, if you wanted to know

$$\int x \sin(x^2) dx$$

you would simply type

```
integral( x*sin(x^2), x )
```

and then you'd learn the answer is

```
-1/2*cos(x^2)
```

Similarly, if you wanted to know

$$\int \frac{x}{x^2 + 1} dx$$

you would simply type

```
integral( x/(x^2 + 1), x )
```

and then you'd learn the answer is

```
1/2*log(x^2 + 1)
```

### More About the +C:

Let's consider the set of functions whose derivative is  $3x^2$ . There's an infinite number of them, including  $x^3 + 5$ ,  $x^3 - 8$ ,  $x^3 + 81$ ,  $x^3 + \sqrt{\pi}$  and so forth. That's why calculus professors insist that you write

$$\int 3x^2 dx = x^3 + C$$

when computing an indefinite integral.

However, Sage does not know about  $+C$ . There's actually a fairly good reason for this. It has to do with the fact that it isn't really easy to make a Python function that represents an infinite family of functions. For example, if I define  $f(x) = \int 3x^2 dx$ , then what should Python say when I ask for  $f(2)$ ? How can Sage know what member of the family I am referring to?

With this in mind, you have to keep track of the  $+C$ s yourself. Correctly handling the  $+C$ s is a key task in one of the projects (the project about ballistic projectiles), in Section 2.5 on Page 79.

### The Definite Integral:

Alternatively, we could put a lower bound of 0 and an upper bound of 1 on that integral, and then get

$$\int_0^1 \frac{x}{x^2 + 1} dx$$

To get Sage to compute that definite integral for us, we type

```
integral( x/(x^2 + 1), x, 0, 1 )
```

and we are the answer

```
1/2*log(2)
```

However, remember that “log” means the natural logarithm, not the common logarithm, as explained on Page 5.

### Impossible Integrals:

The word “impossible” is a dangerous one in mathematics. Some integrals are famously impossible. The two most famous are the Fresnel Integral

$$\int_0^x \sin \frac{\pi t^2}{2} dt$$

which is important in optics, and the Gaussian Integral

$$\int_0^y \frac{2}{\sqrt{\pi}} e^{(-x^2)} dx$$

which is pivotally important in probability. What do we mean by “impossible” here? There is no function, built up of addition, subtraction, multiplication, division, roots, exponents, logarithms, trigonometric functions, and inverse trigonometric functions<sup>9</sup> which will have, as its derivative, either the Fresnel or the Gaussian.

Yet, in applied mathematics, we need to calculate these integrals! In particular, the Gaussian Integral is superbly important in statistics. So what is done, is that the definite integral can be computed by dividing the interval into many, many tiny regions. Lots of narrow rectangles and trapezoids can be used to approximate the curve. The numerical areas of those rectangles and trapezoids are added together, to give a numerical approximation for the integral. (That’s why it is called “numerical integration.”) Each of these rectangles or trapezoids might well have a small amount of error, but when you add up all the areas, the hope is that the errors cancel out a bit. You probably did this technique during your calculus class at some point, or will at some future point.

Even more sophisticated methods for numerical integration include using polynomials instead of trapezoids, and Sage uses extraordinarily intricate techniques to make the best approximation possible. These approximations are an old and deep subject, going back to Simpson’s Rule, where Simpson used tiny parabolas instead of narrow trapezoids. This allowed him to get rather accurate approximations using far fewer computations than the trapezoidal rule would require. Since he was working in the first half of the 1700s, that’s quite an accomplishment.

The above might have been a bit confusing, but the whole point of Sage is that it will worry about these details for you, so long as you know what you want from Sage.

Let’s take a specific example. Consider the following function

$$f(x) = e^{-x^3} \sin(x^2)$$

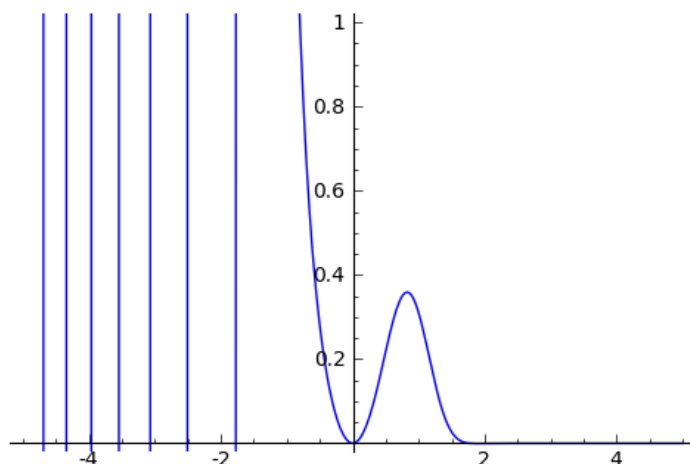
which is an easily understood and continuous function. We can graph it with the commands

```
plot( exp( -x^3 ) * sin(x^2), -5, 5, ymin = 0, ymax=1 )
```

We obtain the following plot

---

<sup>9</sup>We should also include the hyperbolic cousins of the trigonometric functions and inverse trigonometric functions, which you might or might not have been taught about.



This function  $f(x)$  seems interesting, but computing its integral is quite frustrating. I cannot solve that integral with my pencil, and I am willing to wager that you cannot either. Try it if you like.

While it is the case that there is some function  $g(x)$ , somewhere, which has  $f(x) = e^{-x^3} \sin(x^2)$  as its derivative—a logical consequence of some advanced real analysis and the fact that  $f$  is continuous—the practicalities of the matter are that there is no way to write down  $g(x)$  as a formula. To be really precise, there is no function  $g(x)$ , built up of addition, subtraction, multiplication, division, roots, exponents, logarithms, trigonometric functions, and inverse trigonometric functions (as well as their hyperbolic cousins, which you might or might not have been taught about) which will have  $g'(x) = f(x)$ .

In other words  $f(x) = e^{-x^3} \sin(x^2)$  is integrable, but the integral cannot be written in terms of common (and not so common) functions. However, one could produce a  $g(x)$  made with an infinite series in Big-Sigma notation that would have  $g'(x) = f(x)$ .

Returning now to Sage, since

$$\int e^{-x^3} \sin(x^2) dx$$

does not have a nice expression, then when you type

```
integral( exp(-x^3)*sin(x^2),x)
```

the Sage replies not with an answer, but instead with

```
integrate(e^(-x^3)*sin(x^2), x)
```

which represents an admission of defeat. However, have no fear, because we can calculate numerical answers approximately—with very high accuracy—and we'll learn about that momentarily.

**Numerical Integration in Sage:**

A nefarious integral, famous among calculus teachers, is

$$\int t^{20} e^t dt$$

where the 20 could be any decently large number. This comes up in explaining what Euler's gamma function is, but we aren't too concerned with that right now. In any case, we could just type

```
integral( (t^20)*(e^t), t )
```

but then we get back

```
(t^20 - 20*t^19 + 380*t^18 - 6840*t^17 + 116280*t^16 - 1860480*t^15
+ 27907200*t^14 - 390700800*t^13 + 5079110400*t^12 - 60949324800*t^11
+ 670442572800*t^10 - 6704425728000*t^9 + 60339831552000*t^8
- 482718652416000*t^7 + 3379030566912000*t^6 - 20274183401472000*t^5
+ 101370917007360000*t^4 - 405483668029440000*t^3
+ 1216451004088320000*t^2 - 2432902008176640000*t
+ 2432902008176640000)*e^t
```

which is large enough that it isn't clear how to get an idea of what that means. If instead, we had certain bounds, such as to calculate

$$\int_2^3 t^{20} e^t dt$$

then we would type

```
integral( (t^20)*(e^t), t, 2, 3 )
```

and get instead

```
-329257482363600896*e^2 + 121127059051462881*e^3
```

which is getting toward an answer! If you really want a numerical answer then you should use our old trick, the command  $N()$ , and type

```
N( integral( (t^20)*(e^t), t, 2, 3 ) )
```

which would return the number

```
8.79797452800000e9
```

and that means  $8.79797452800000 \times 10^9$ . This highlights the difference and similarities of the various types of answer. If you've forgotten about  $N()$ , see Page 3.

That number which we just calculated looks like an integer, because it ends with 528 when you carry out the  $10^9$  part—but when we look at the exact answer, we can see that no, it is not an integer. We know that because  $e^2$  and  $e^3$  are involved, and  $e$  is an irrational number. Whenever dealing with computations, it is critical to remember what is an approximation, and what is exact. Irrational number can be expressed exactly on occasion (like we just did here, using  $e^2$  and  $e^3$ ) but that is the exception and not the rule. In any case, this paragraph has nothing to do with Sage, therefore we return to the topic at hand.

Speaking of approximations, you can also jump to the numerical integral very rapidly. I'm sure we can both calculate

$$\int_0^1 (x^3 - x) \, dx = \left( \frac{1}{4}x^4 - \frac{1}{2}x^2 \right) \Big|_0^1 = \left( \frac{1}{4}1^4 - \frac{1}{2}1^2 \right) - \left( \frac{1}{4}0^4 - \frac{1}{2}0^2 \right) = \frac{1}{4} - \frac{1}{2} = -\frac{1}{4}$$

However, if you wanted to ask Sage to calculate that integral numerically, then you must type

```
numerical_integral( x^3 - x, 0, 1)
```

which will output

```
(-0.24999999999999997, 2.775557561562891e-15)
```

where the first number is the best guess Sage has for the answer, while the second number is the uncertainty. In this case, the uncertainty is 2.77 quadrillionths—which is very impressive.

Of course, Sage can do this integral exactly as well, using the commands we learned moments ago. What is interesting about numerical integration are the cases when you cannot do the indefinite integral, because like the Fresnel Integral above it cannot be written, but you can find good numerical estimates. Consider again

$$\int_1^3 e^{-x^3} \sin(x^2) \, dx$$

but now with bounds, so that it can be done numerically. We would type

```
numerical_integral( exp(-x^3)*sin(x^2),1,3)
```

and that returns back

```
(0.077997011262087815, 8.6840496851995037e-16)
```

with (as before) the first number being the answer, and the second the uncertainty. In this case, the uncertainty is 868 quintillionths—which is very impressive. Another way to say it is that the uncertainty is (1/1152) trillionths.

### Integration by Partial Fractions:

One thing that is very important in problems that arise from *Differential Equations* and *Calculus II* is the question of integration by partial fractions. If you want to know the partial fraction break-down of

$$\frac{x^3 - x}{x^2 + 5x + 6}$$

then you must do

```
f(x)=(x^3-x)/(x^2+5*x+6)
```

followed by

```
f.partial_fraction()
```

and get the answer

```
x - 6/(x + 2) + 24/(x + 3) - 5
```



which is correct. In fact, we just calculated

$$\frac{x^3 - x}{x^2 + 5x + 6} = x - \frac{6}{x+2} + \frac{24}{x+3} - 5$$

However, you can also do the shortcut and ask instead

```
integral( (x^3-x)/(x^2+5*x+6), x )
```

which will result in

```
1/2*x^2 - 5*x - 6*log(x + 2) + 24*log(x + 3)
```

### Improper Integrals:

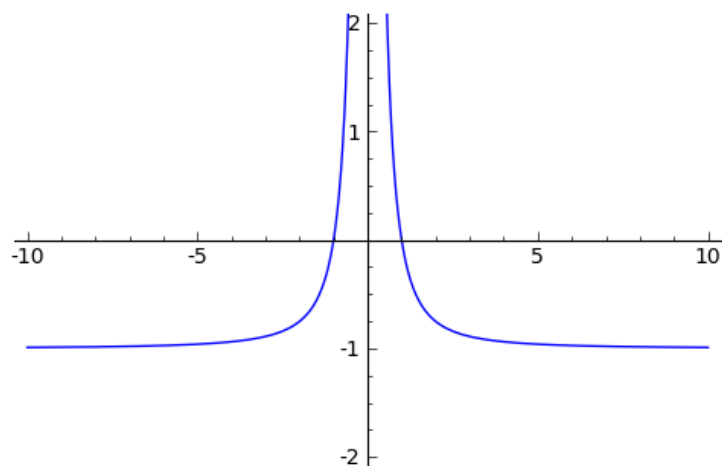
While the following integral is mathematically valid

$$\int \left( -1 + \frac{1}{x^2} \right) dx = -x - \frac{1}{x} + C$$

and the right-hand side can be evaluated at  $x = -1$  and  $x = 1$ , there is no finite answer to

$$\int_{-1}^1 \left( -1 + \frac{1}{x^2} \right) dx$$

because this improper integral is divergent. The graph will help reveal why that is the case. Here is the plot of  $-1 + 1/x^2$



generated by

```
plot( -1+1/x^2, -10, 10, ymin=-2, ymax=2 )
```

Yet, Sage knows this and will say

```
-x - 1/x
```

in response to

```
integral(-1+1/x^2,x)
```

but instead will respond

```
ValueError: Integral is divergent.
```

to the inquiry

`integral(-1+1/x^2,x,-1,1)`

If you'd like a proof of the fact that the above integral is divergent, consider that

$$\int_{-1}^{-1} \left(-1 + \frac{1}{x^2}\right) dx = \int_{-1}^0 \left(-1 + \frac{1}{x^2}\right) dx + \int_0^1 \left(-1 + \frac{1}{x^2}\right) dx$$

Let's consider the integral above, from 0 to 1, by means of computing that integral from  $a$  to 1, where  $a$  will be slightly more than 0.

$$\begin{aligned} \int_a^1 \left(-1 + \frac{1}{x^2}\right) &= \left(-x - \frac{1}{x}\right) \Big|_a^1 \\ &= \left(-1 - \frac{1}{1}\right) - \left(-a - \frac{1}{a}\right) \\ &= a - 2 + \frac{1}{a} \end{aligned}$$

As you can see, when  $a$  approaches 0 from the positive real numbers, the value of  $a - 2 + 1/a$  becomes huge. More precisely, we can write

$$\lim_{a \rightarrow 0^+} \left(a - 2 + \frac{1}{a}\right) = \infty$$

but then that also means

$$\lim_{a \rightarrow 0^+} \int_a^1 \left(-1 + \frac{1}{x^2}\right) dx = \infty$$

and, as it turns out, the same is true for the integral from -1 to 0 as well. Therefore, the original integral is equal to  $\infty + \infty = \infty$ .

### More Improper Integrals:

Another type of improper integral is

$$\int_2^{\infty} \frac{1}{x^2} dx$$

and the way to write that in Sage is to type

`integral(1/x^2,x,2,oo)`

to which Sage responds 1/2, or another example is

$$\int_{-\infty}^{\infty} e^{-x^2} dx$$

which in Sage is

`integral(exp(-x^2),x,-oo,oo)`

and it gives the correct response of  $\sqrt{\pi}$ .

The idea is simply that `oo` is a special code for “infinity.” It is two letter o's, and the idea is that if you squint, `oo` looks like an infinity symbol.

**The Function erf and Integrals:**

Among mathematicians and especially among statisticians, the integral

$$\int_0^y \frac{2}{\sqrt{\pi}} e^{(-x^2)} dx = \text{erf}(y)$$

is extraordinarily important. This is the Gaussian Integral that I told you about earlier; in particular, I said that this integral cannot be found using the elementary functions of mathematics. It is so important that it has a name, and that name is **erf**. While being friendly and pronounceable, **erf** stands for the “Error Function.”

Thus if you type into Sage

```
integral( (2/sqrt(pi)) * exp(-x^2), x, -oo, 2)
```

it will respond

```
(sqrt(pi)*erf(2) + sqrt(pi))/sqrt(pi)
```

which is correct because

$$\begin{aligned} \int_{-\infty}^2 \frac{2}{\sqrt{\pi}} e^{(-x^2)} dx &= \int_{-\infty}^0 \frac{2}{\sqrt{\pi}} e^{(-x^2)} dx + \int_0^2 \frac{2}{\sqrt{\pi}} e^{(-x^2)} dx \\ &= 1 + \text{erf}(2) = \frac{\sqrt{\pi}}{\sqrt{\pi}} (1 + \text{erf}(2)) = \frac{\sqrt{\pi} \text{erf}(2) + \sqrt{\pi}}{\sqrt{\pi}} \end{aligned}$$

Perhaps you might find that  $1 + \text{erf}(2)$  is a more compact and simpler answer. I would be inclined to agree. I’m not quite sure why Sage does not simply its final answer further.

In any case, the point is that there are some functions out there which do not have integrals writable using elementary functions, but, which Sage calculates anyway, using the **erf** function.

**The Assume Command, and Integrals**

This example was identified by Joseph Loeffler, who brought it to my attention. Let’s say that you wanted to evaluate the integral

$$\int_1^x \frac{t^3 + 30}{t} dt$$

and that you typed, accordingly, the Sage commands:

```
var("t")
integrate( (t^3+30)/t, t, 1, x)
```

(By the way, **integrate** is a synonym for **integral**.) As it turns out, you’d receive back a response that is a huge error message. We must remember that in Sage, the last lines of an error message are the ones that are important. The last lines are as follows:

```
ValueError: Computation failed since Maxima requested additional
constraints; using the 'assume' command before integral
evaluation *may* help (example of legal syntax is 'assume(x-1>0)',
see 'assume?' for more details)
```

Is  $x-1$  positive, negative or zero?

The complaint here, sent to Sage from Maxima, is justified because the function  $f(t) = (t^3 + 30)/t$  will explode at  $t = 0$ , because it attempts to divide by zero. The integral is therefore improper if 0 is found between 1 and  $x$ . However, if  $x - 1 > 0$ , or in plain English, if  $x > 1$ , then that's clearly not going to happen. Therefore, Sage is very justified in asking us to explicitly state this assumption.

Let's follow Sage's recommended course of action by adding the `assume` command before our `integrate` command. We now have

```
var("t")
assume( x>1 )
integrate( (t^3+30)/t, t, 1, x)
```

which produces the response

```
1/3*x^3 + 30*log(x) - 1/3
```

as expected.

The `assume` command is rare enough that many seasoned Sage users do not know that it exists. It is unlikely to come up in your work. However, we'll see another example of it on Page 184, in a summation, and on Page 202 in an integral related to a Laplace Transform.

### 1.13. Sharing the Results of Your Work

In this section, I'll share with you seven effective ways of sharing the results of your work in Sage. These can be great for scientific collaboration, submitting work to a faculty member, getting help from a friend, or just explaining a math concept to someone. My two most favorite methods are accessed via the "Share" button on the Sage Cell Server screen.

When you have work in the Sage Cell Server, you can click on the "Share" button. Then you will see options for making a short temporary link, a permalink, and a "2D bar code," also called a "QR code." The permalink is my favorite. It will generate an enormous URL, and prove to you that the new URL works by reloading the page with that URL. Then you should highlight that URL from the web-browser, from the spot where you would normally type <https://sagecell.sagemath.org/>, taking great care to make sure that you've highlighted the whole thing. This URL can now be used in documents, in emails, in chats, or even on Facebook. The URL actually encodes the Sage code in the window, and thus your code is not stored anywhere and therefore cannot be lost, deleted, or destroyed. As long as you retain a complete and unmodified URL, then that URL will always work, even years later.

The permalink is long and ugly. This should not be surprising, because it actually encodes your entire Sage block of code. In fact, the permalink is so long that it can cause problems in some browsers or email processors. In almost all cases, it is longer than 140 characters and thus cannot be sent in

a text message or be tweeted. The short temporary link, on the other hand, is shorter and less intimidating to a human. Being shorter, it is less likely to be truncated in transit when sent by various means. However, it does not contain a complete copy of your code. Instead, your code is stored on the server, and the URL retrieves it. There is no guarantee for how long the short temporary link will work. Perhaps a day, perhaps a year, or perhaps forever but perhaps only an hour. In my experience, they often work several days after being generated. Overall, I'd have to say that if the length and ugliness are not a problem, then the permalink should always be used and the short link should never be used.

Those are my two favorite methods of sharing Sage code. The third method I'd like to share with you is the large 2D barcode produced when you click “share.” This actually encodes the permalink and thus will work with all smartphones that can read those 2D barcodes. Most phones will require you to install an app for the smartphone to be able to handle those barcodes, which are sometimes called “QR codes,” “Quick Response Codes,” or “matrix barcodes.” On my phone, the app<sup>10</sup> was called “QR Code Reader,” and was free. If you photograph the barcode using one of those apps and the camera inside your smartphone, then it should open up a web browser and present the same results as the permalink will. You don't even need to click “Evaluate”! Here is a sample 2D barcode:



The fourth method is when you've made a plot (as we learned in Section 1.4 on Page 8). If you'd like to include this image in any sort of document, or email it to someone, then just save the image file from your web browser—in the same way that you'd save any other image. In most browsers, this is a right-click followed by selecting “Save Image As...,” and entering in a filename at the appropriate spot. This image will be in the

---

<sup>10</sup>You really do need the app, however. Otherwise you just get a photograph of the QR Code, which is totally unexciting.

“portable network graphic” or `*.png` format. It will work in nearly all programs that allow the importing of an image. Alternatively, some high-end mathematics and physics journals prefer “encapsulated postscript” or `*.eps` files. Those can be generated by

```
P = plot( x^3-x, -2, 2 )
P.save("myplot.eps")
```

which will give you a link that will start the downloading of `myplot.eps` when clicked upon.

The fifth method is just highlighting the Sage code, copying, and then pasting it into an email. Sometimes I will paste it into SageMathCloud, or my university’s course management software, D2L. Emailing code can save time, and it is easy to post code in this way.

The sixth method is to print a screenshot of the Sage Cell Server. Using a bit of scrolling around and resizing, one can often cause the entire Sage block of code and its output to be visible at the same time. Then you can use your browser’s print command (often under the file menu) to print the webpage just like you’d print any other webpage. (Often the web browser’s print command is under the “File” menu.) This is often how I ask students in my classes to submit a homework problem that has been done in Sage. Last but not least, the seventh method is to follow the steps for printing a screenshot of the Sage Cell Server, and instead of printing it as a physical paper document, to select “Save as a PDF” from the web browser’s print menu. This PDF is essentially identical as a document to what you’d get from printing, except that it is an electronic copy, rather than a paper copy.

Scientific collaboration is one of the great joys of working in a STEM<sup>11</sup> subject. I hope that you will have frequent opportunities to share not only your ideas, but the hardcore mathematical and quantitative details of your work, with others. Since Sage has been developed by hundreds of people scattered across the globe, it is unsurprising that there many features built-in to Sage to enable sharing. SageMathCloud has even more features, including organized projects, sharing on an invitation-only basis, and live-chat features. You can learn about those on the SageMathCloud website.

---

<sup>11</sup>STEM: Science, Technology, Engineering and Mathematics.